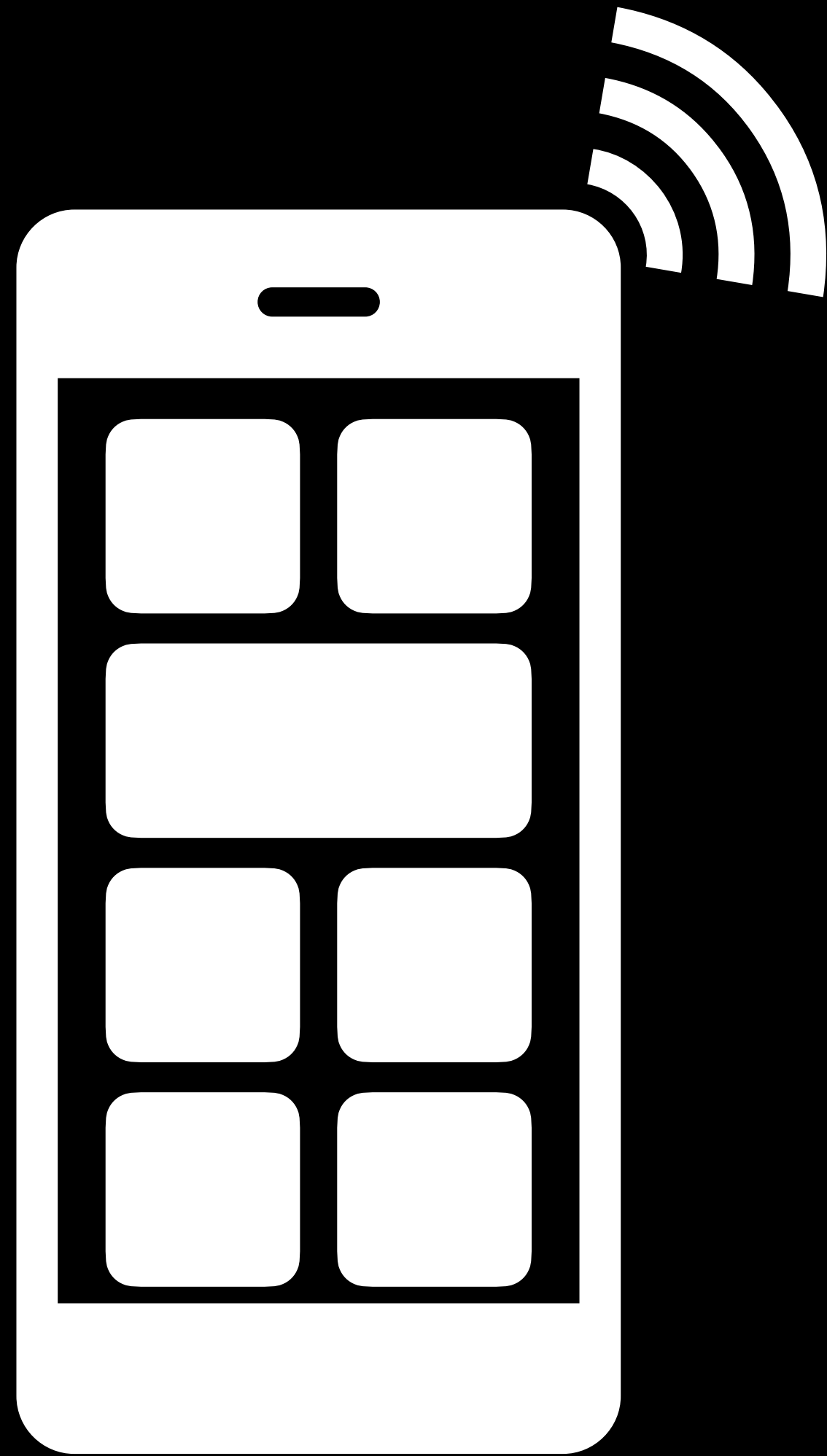


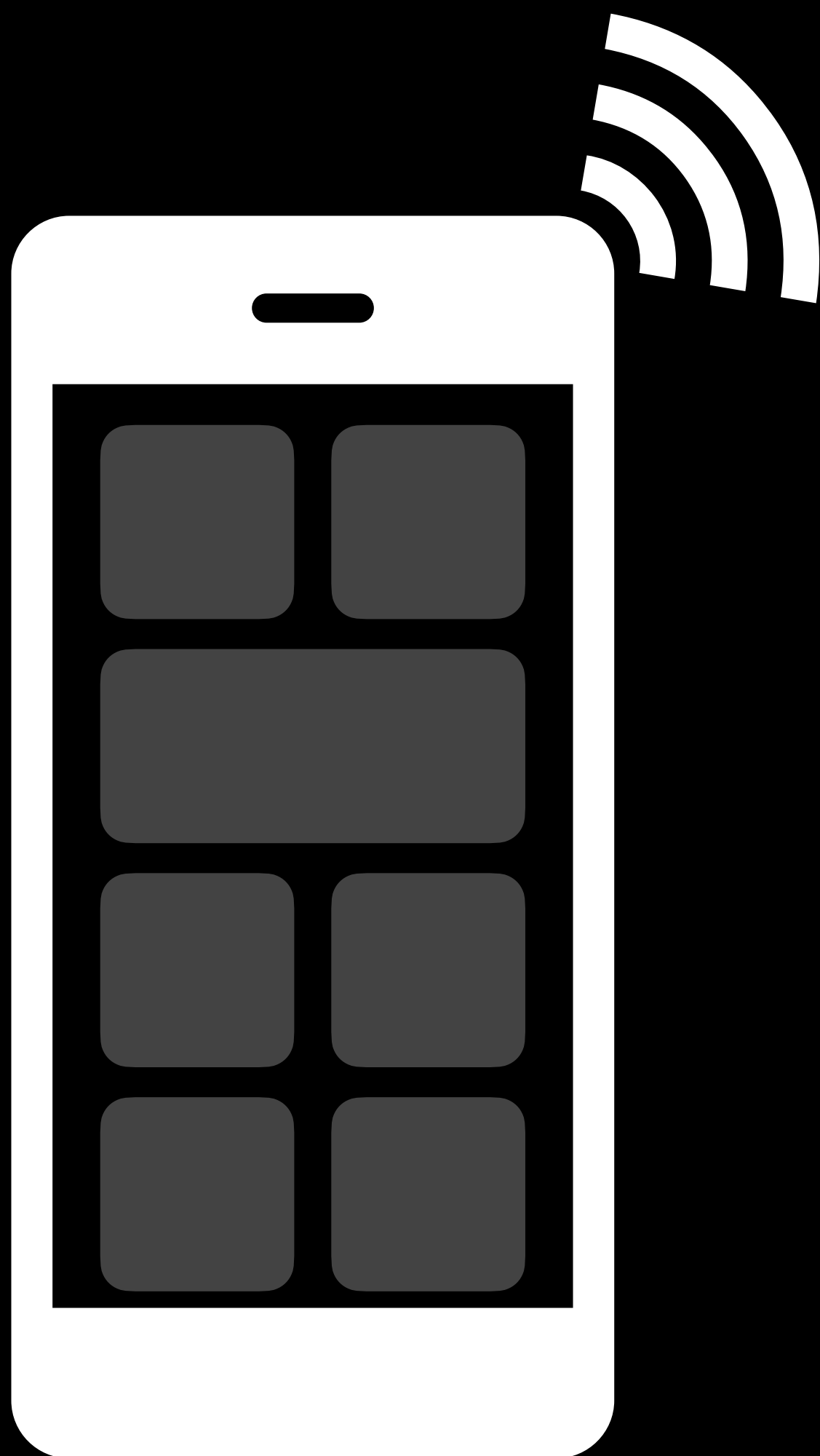
Offline Apps using SQLiteData

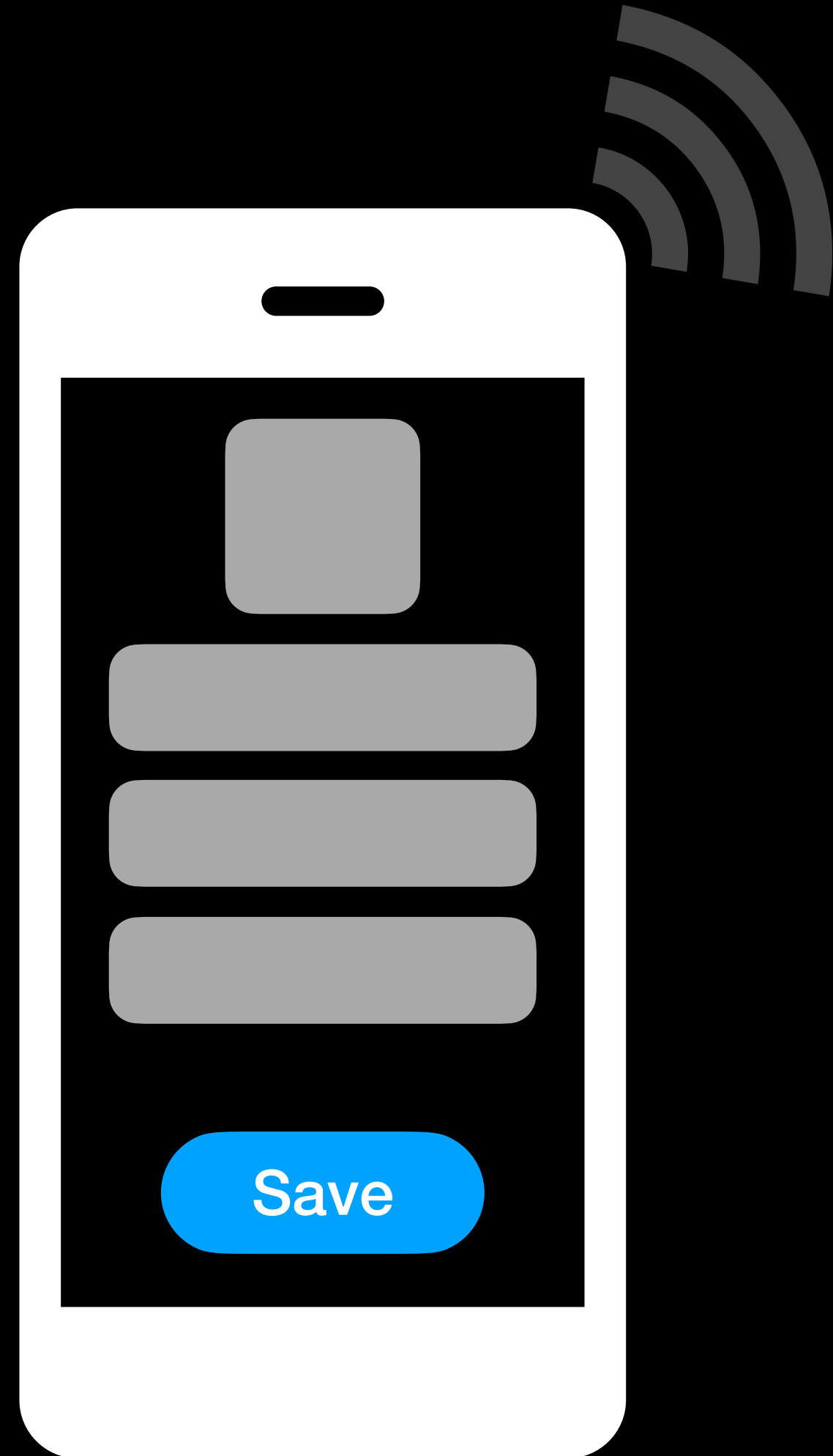
SwiftData Alternative

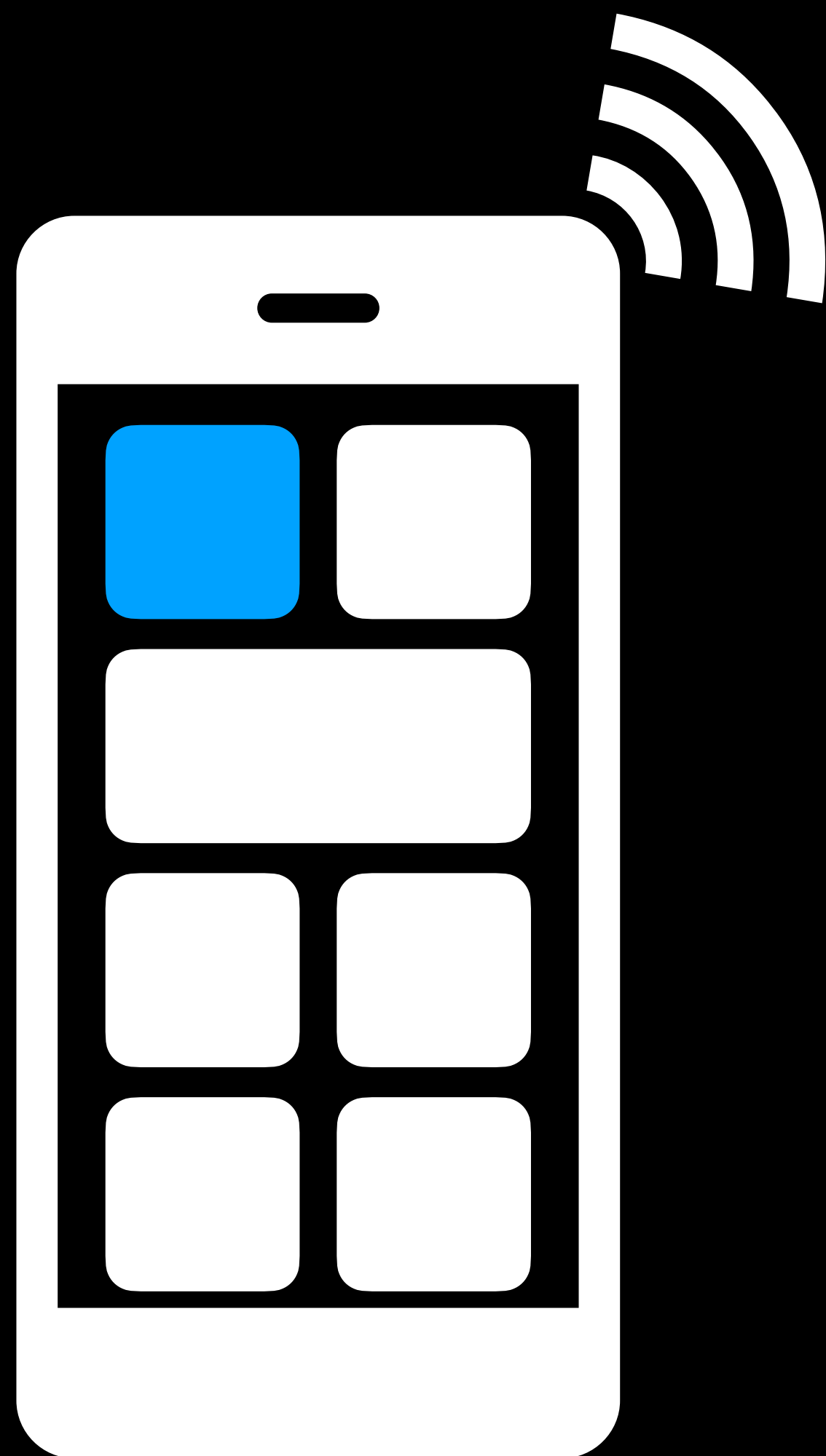
Ratnesh Jain

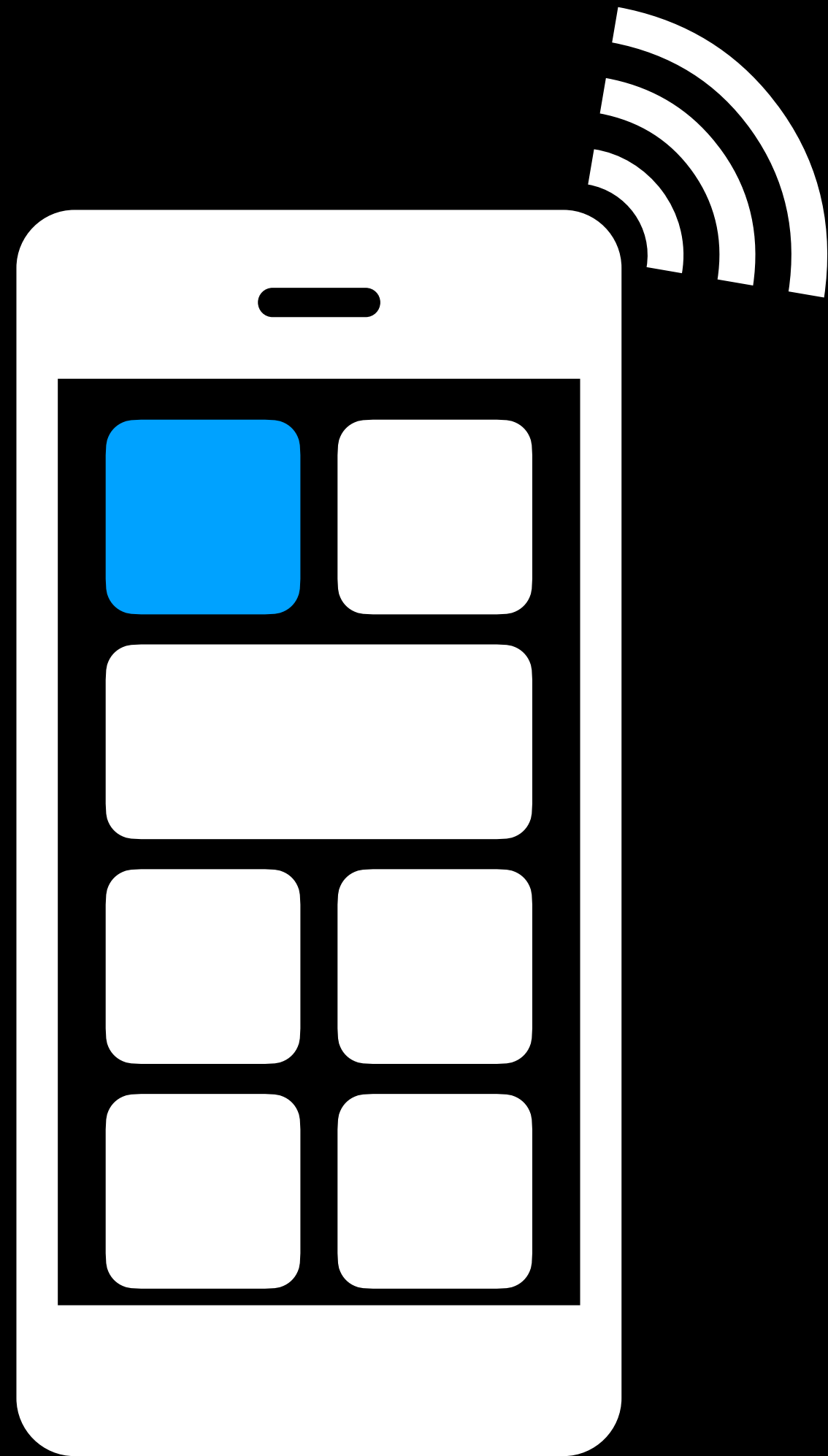
Offline Apps

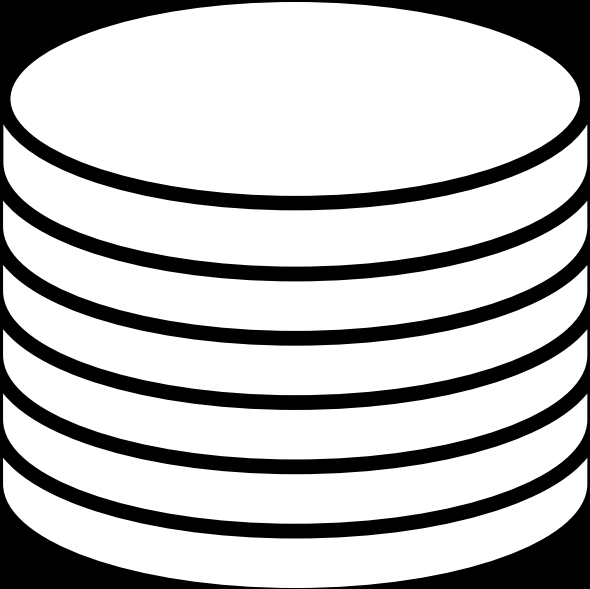
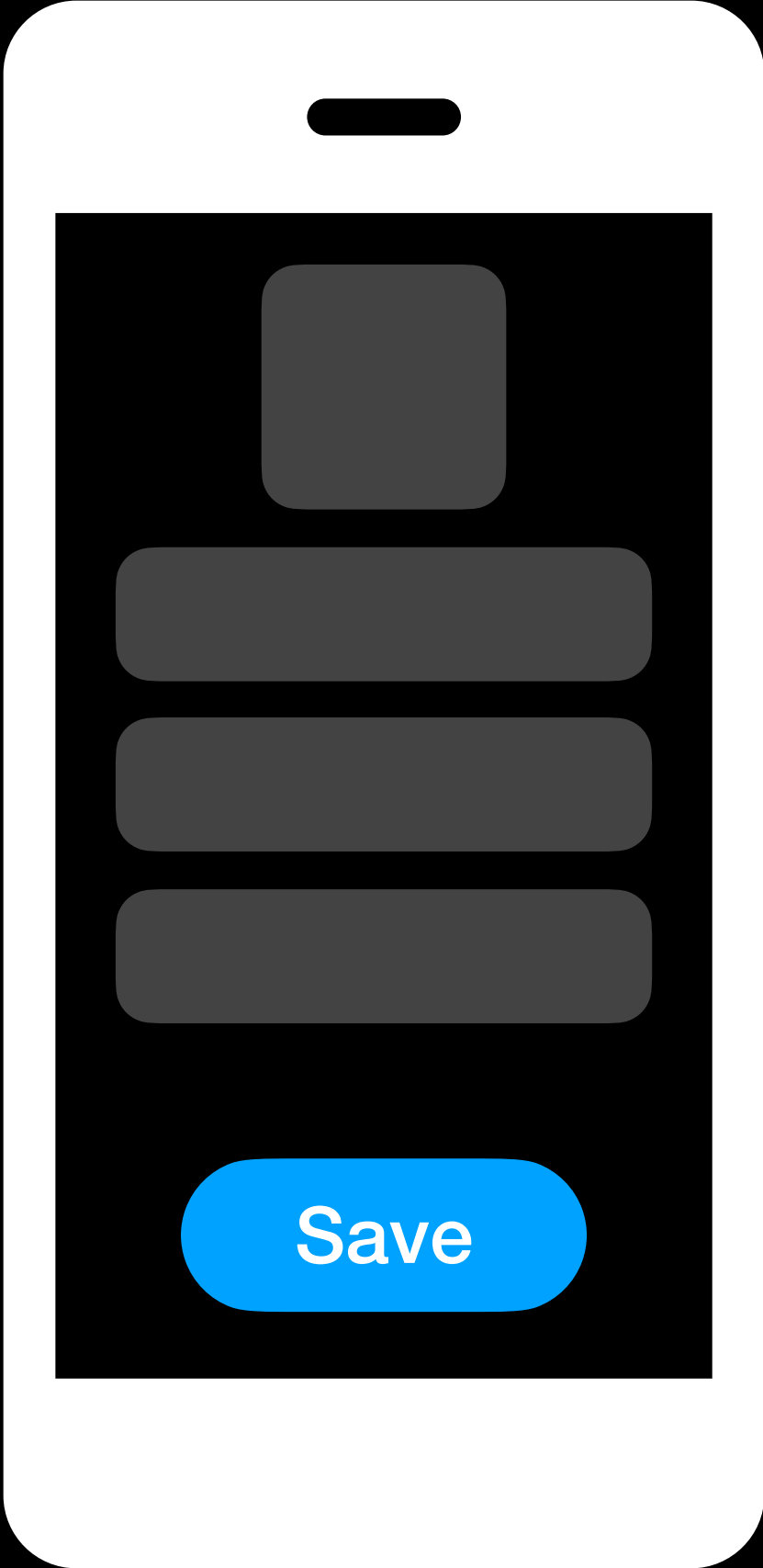




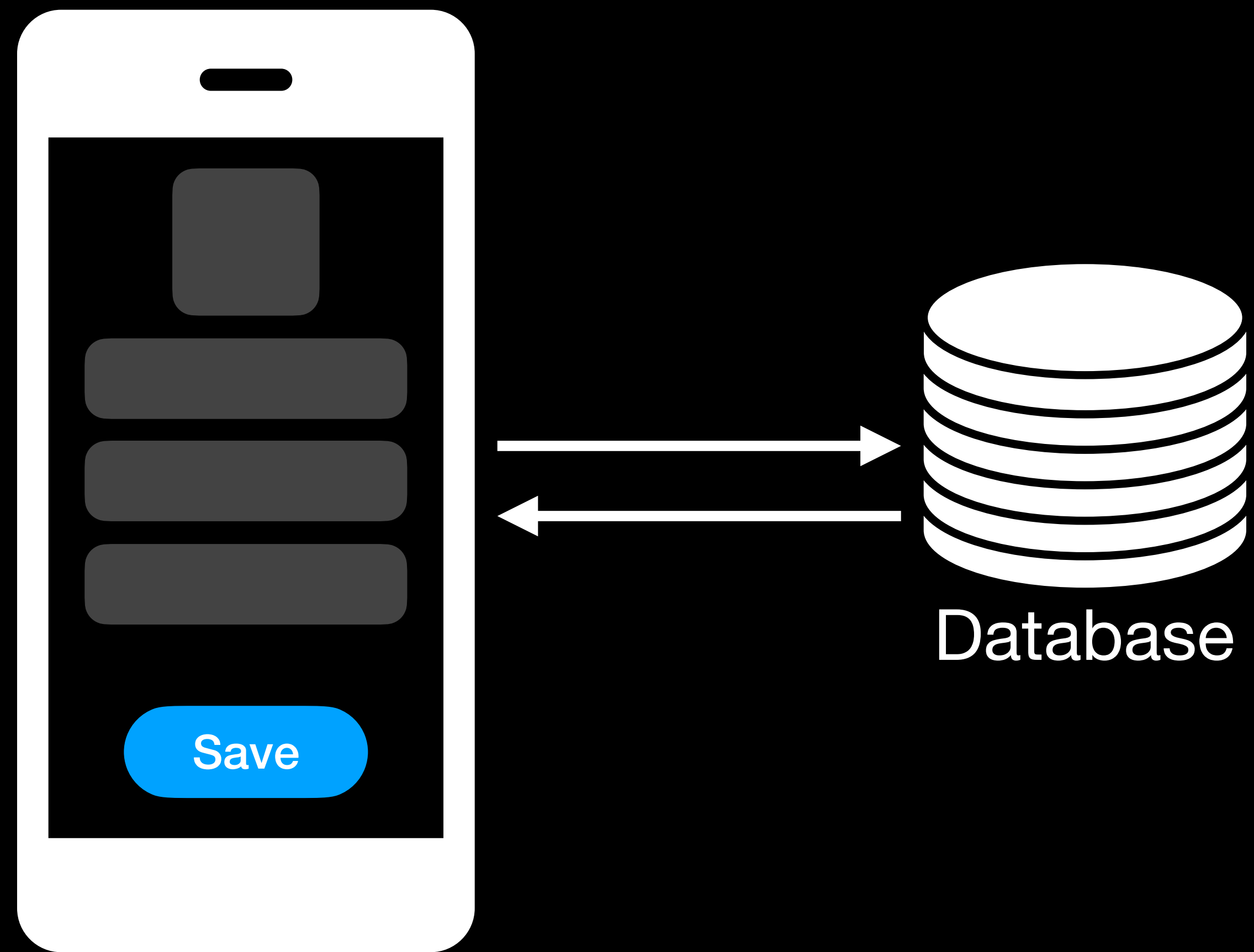


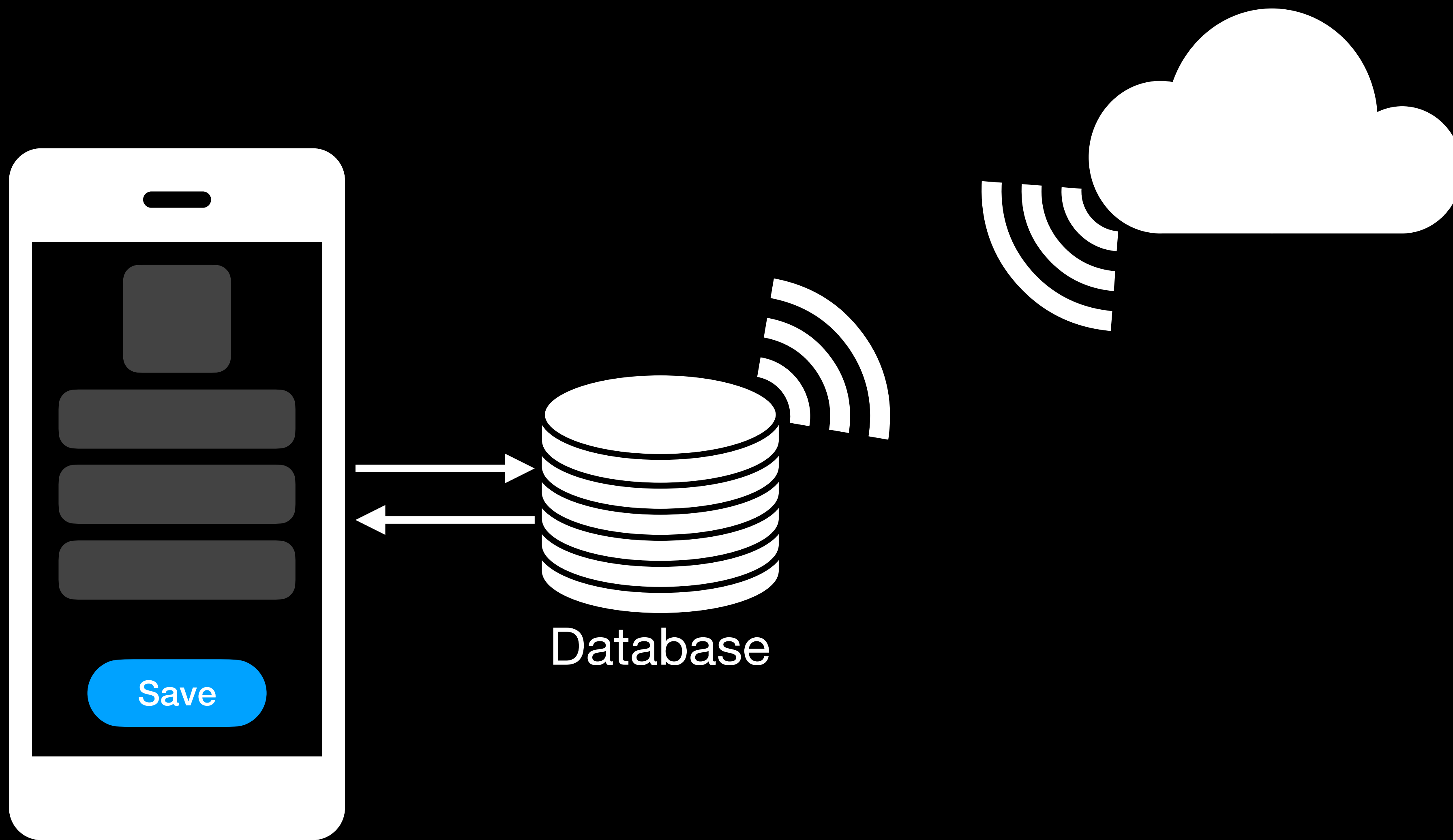


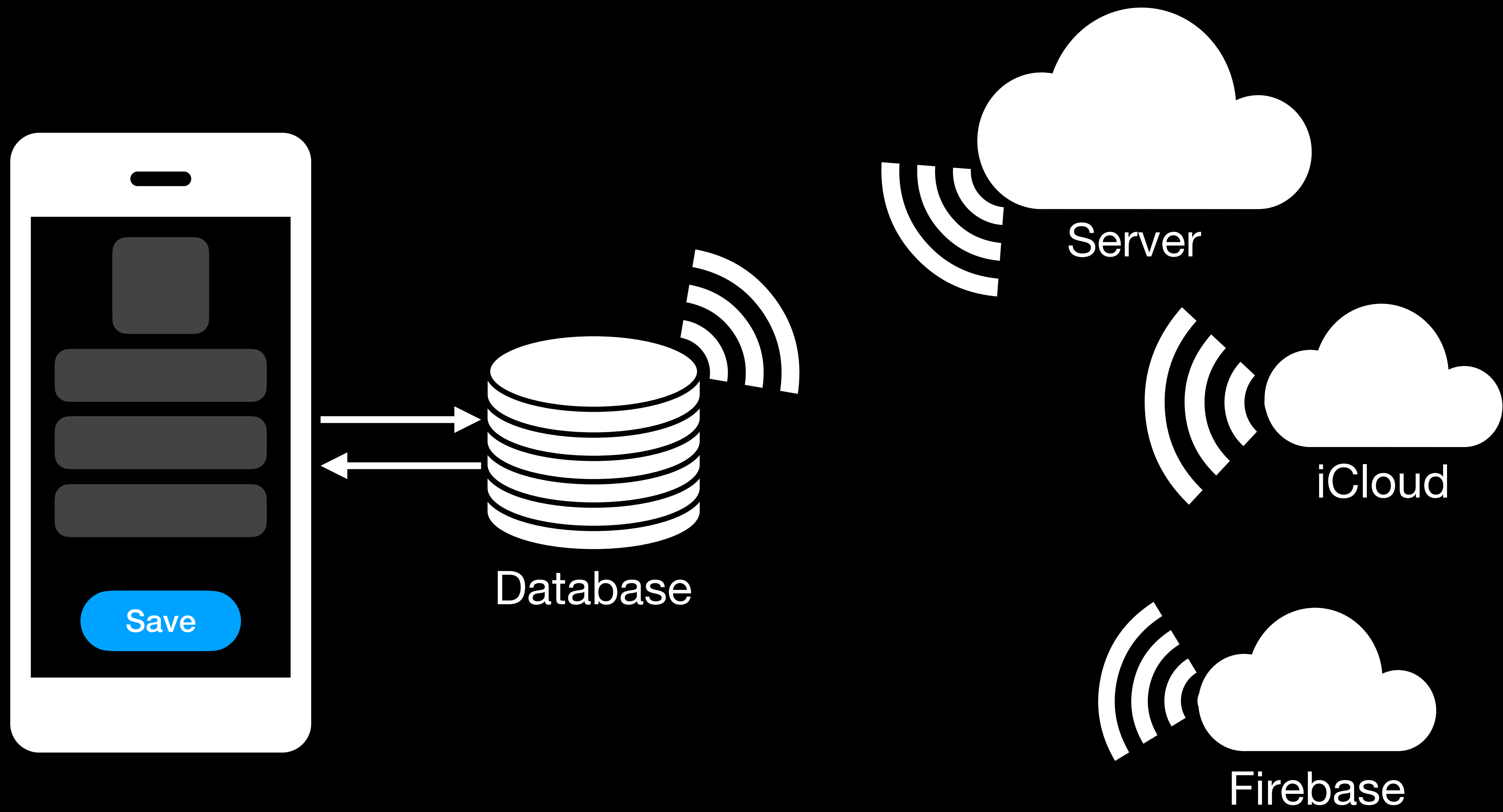


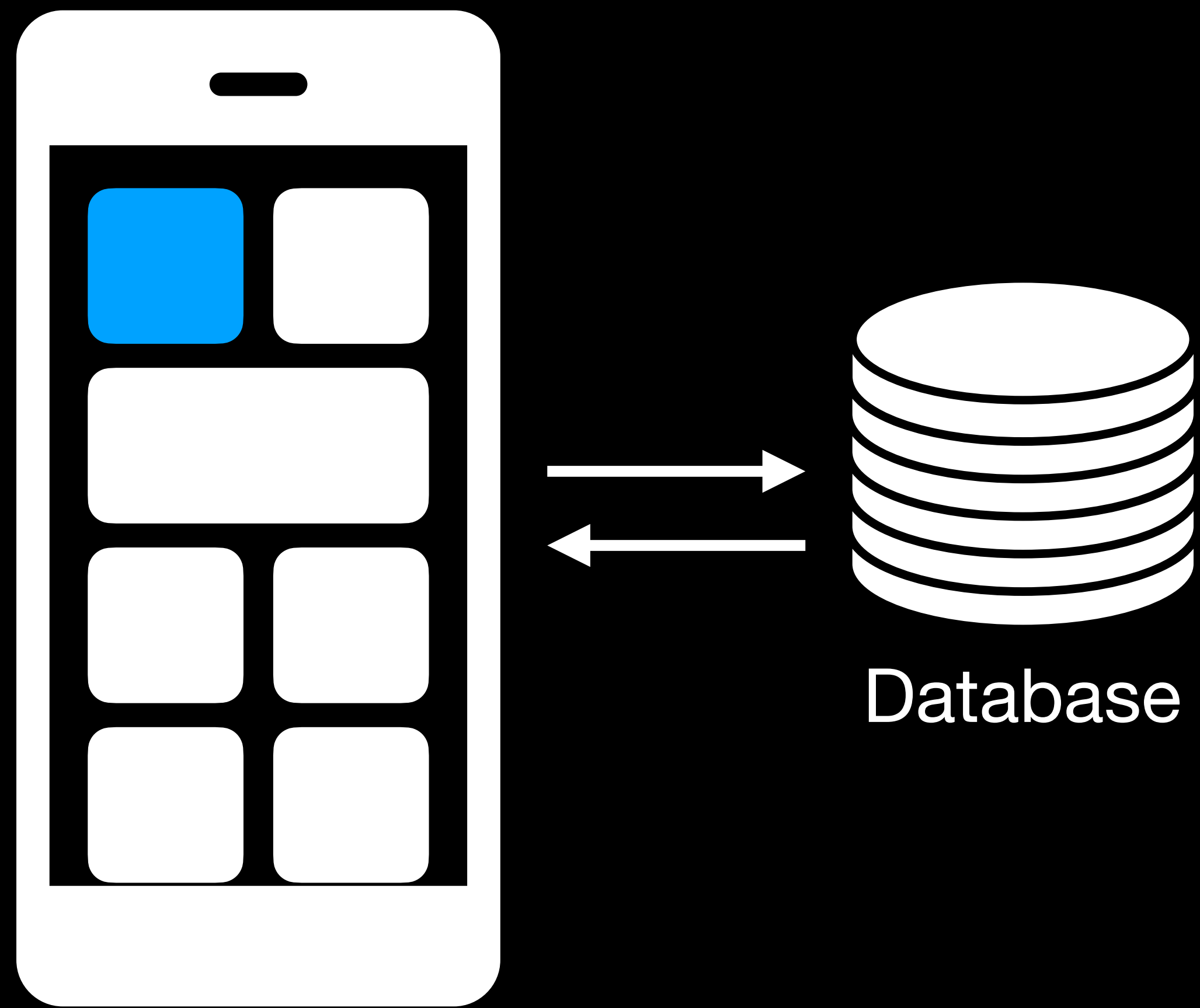


Database

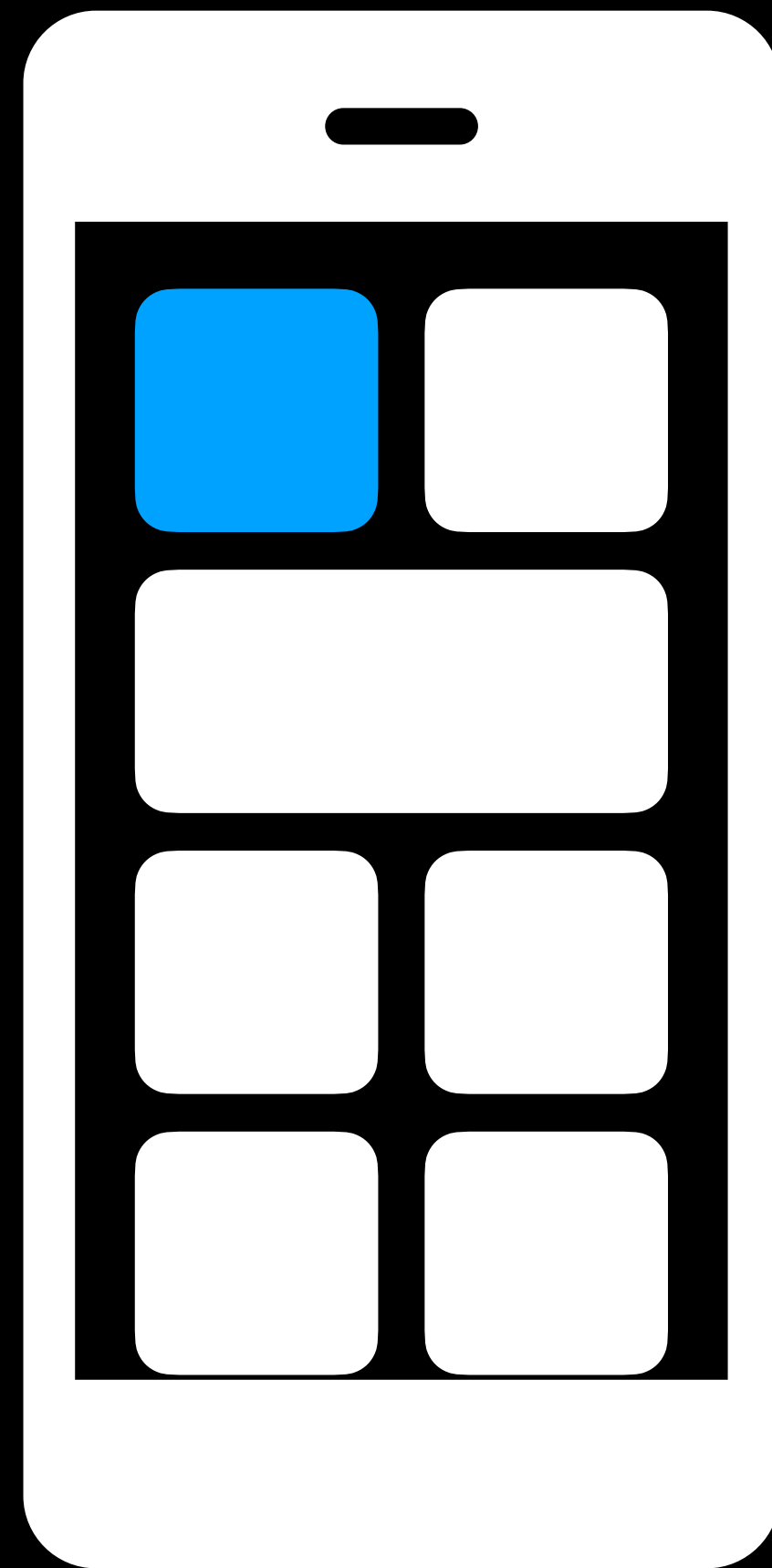








Single Source of Truth



Persistence Framework

Persistence Frameworks



CoreData



SwiftData



GRDB

CoreData Setup

iOS 10+



```
open class PersistentContainer: NSPersistentContainer {  
    ...  
}  
  
public enum PersistentModelStore {  
    public static let container: PersistentContainer = {  
        guard let modelURL = Bundle.main.url(forResource: "PersistentModels", withExtension: "momd") else { fatalError() }  
        guard let model = NSManagedObjectModel(contentsOf: modelURL) else { fatalError() }  
        let container = PersistentContainer(name:"PersistentModels", managedObjectModel: model)  
        container.loadPersistentStores(completionHandler: { (storeDescription, error) in  
            if let error = error as NSError? {  
                print("Unresolved error \(error), \(error.userInfo)")  
            }  
        })  
        return container  
    }()  
}
```

CoreData Setup

iOS 10+



```
open class PersistentContainer: NSPersistentContainer {  
    ...  
}  
  
public enum PersistentModelStore {  
    public static let container: PersistentContainer {  
        guard let modelURL = Bundle.main.url(forResource: "PersistenceModels", withExtension: "momd") else { fatalError() }  
        guard let model = NSManagedObjectModel(contentsOf: modelURL) else { fatalError() }  
        let container = PersistentContainer(managedObjectModel: model)  
        container.loadPersistentStores(completionHandler: { (storeDescription, error) in  
            if let error = error as NSError? {  
                print("Unresolved error: \(error.localizedDescription)")  
            }  
        })  
        return container  
    }()  
}
```

PersistenceModels.momd

PersistentContainer

ManagedObjectModel

URL

ManagedObjectContext

CoreData Setup

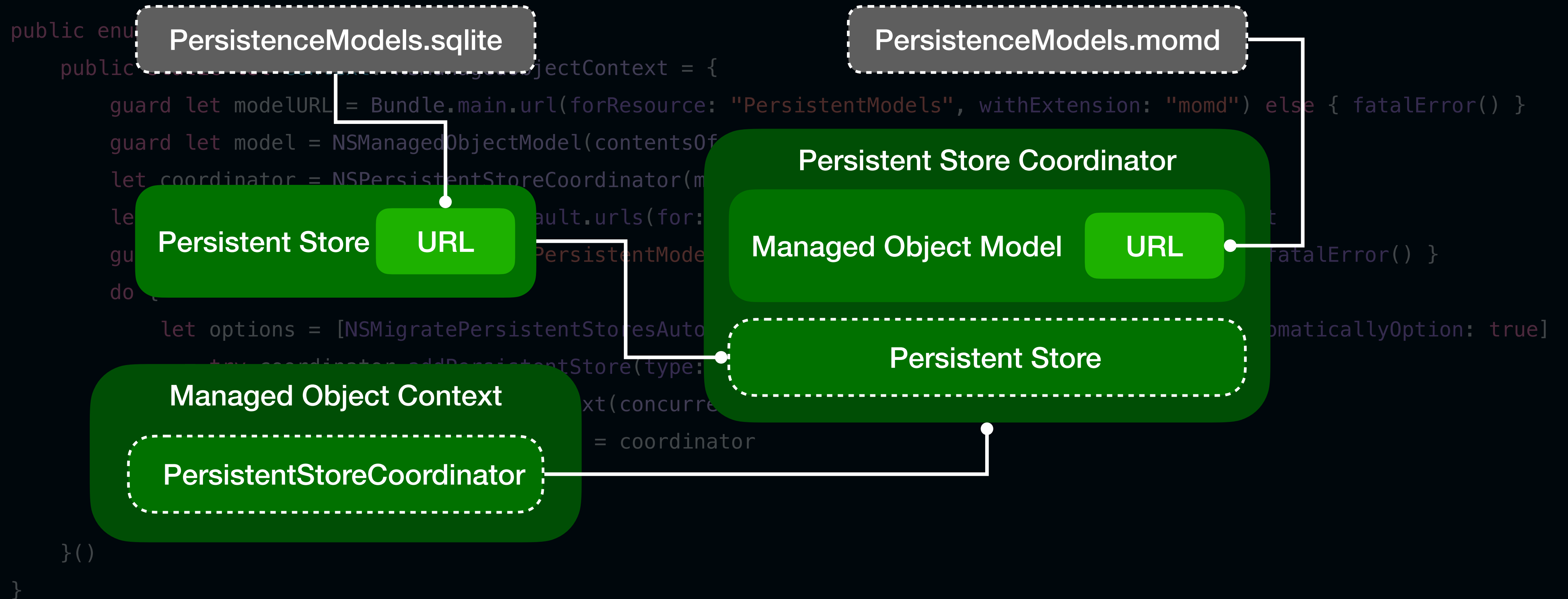
iOS 3+



```
public enum PersistentModelStore {
    public static let context: NSManagedObjectContext = {
        guard let modelURL = Bundle.main.url(forResource: "PersistentModels", withExtension: "momd") else { fatalError() }
        guard let model = NSManagedObjectModel(contentsOf: modelURL) else { fatalError() }
        let coordinator = NSPersistentStoreCoordinator(managedObjectModel: model)
        let docDirectory = FileManager.default.urls(for: .documentDirectory, in: .userDomainMask).last
        guard let storeURL = URL(string: "PersistentModels.sqlite", relativeTo: docDirectory) else { fatalError() }
        do {
            let options = [NSMigratePersistentStoresAutomaticallyOption: true, NSInferMappingModelAutomaticallyOption: true]
            _ = try coordinator.addPersistentStore(type: .sqlite, at: storeURL, options: options)
            let context = NSManagedObjectContext(concurrencyType: .mainQueueConcurrencyType)
            context.persistentStoreCoordinator = coordinator
        } catch { fatalError() }
        return context
    }()
}
```

CoreData Setup

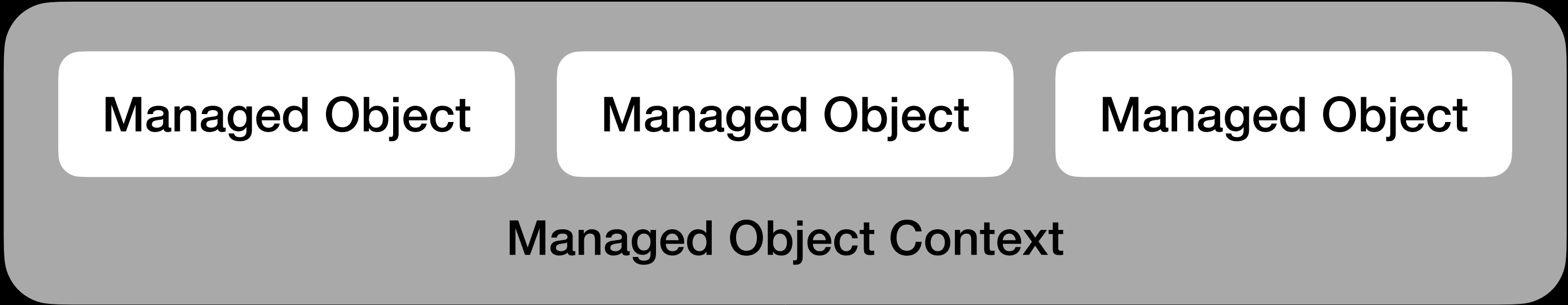
iOS 3+



CoreData



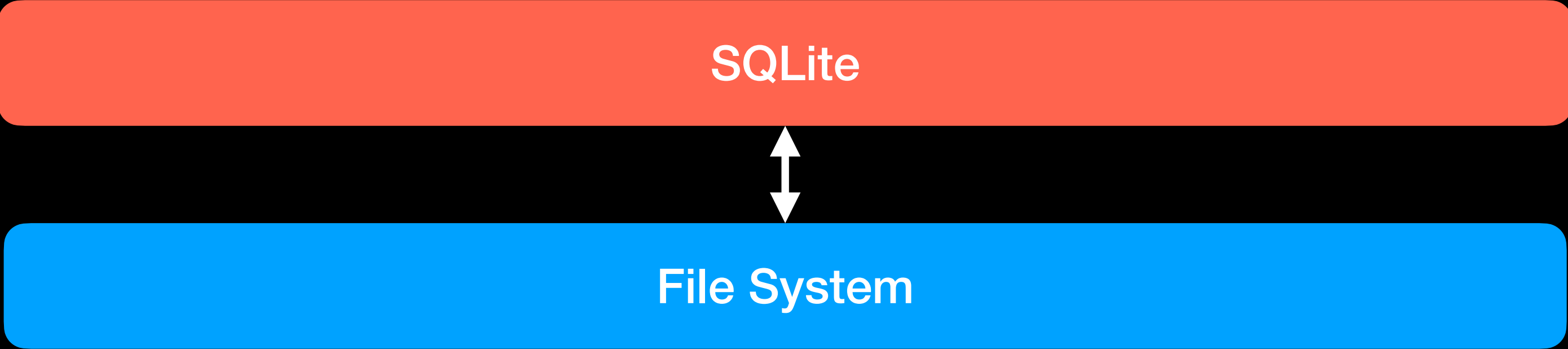
Context Tier

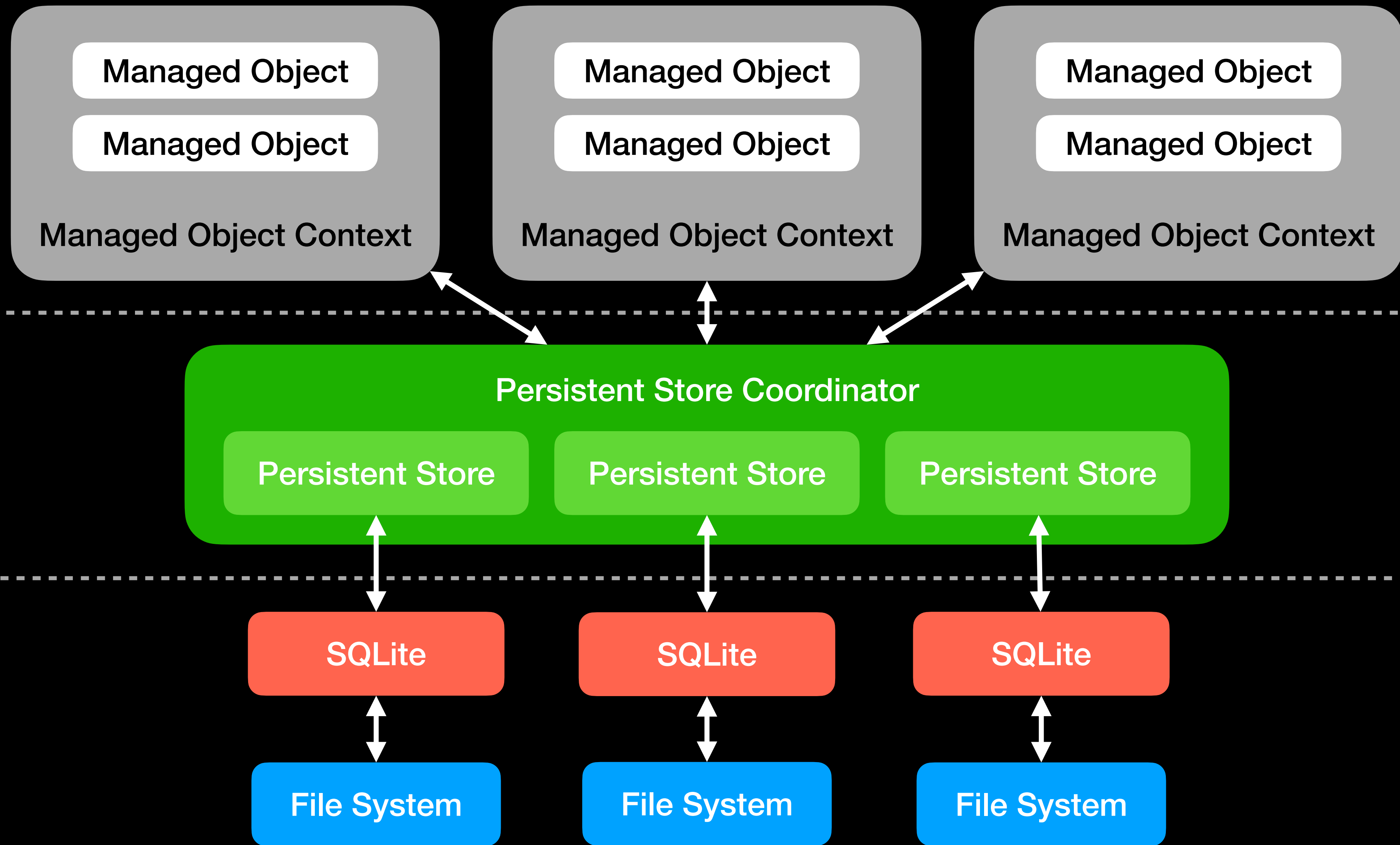


Coordinator Tier



SQLite Tier





CoreData



Application

Context

Coordinator
(with Persistent
Store)

SQLite

Save

execute(_ request:)

execute(_ request: with context:)

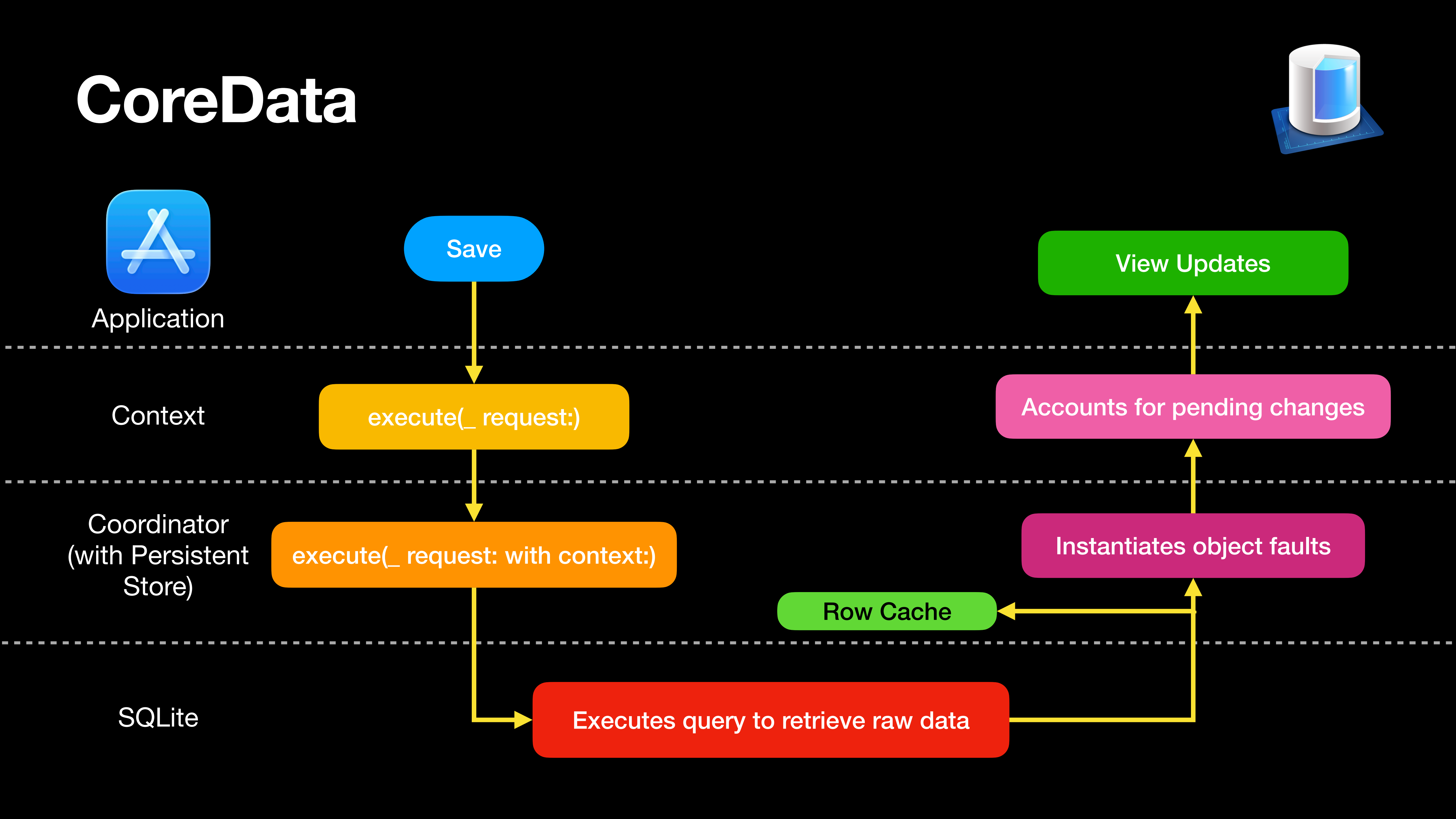
Row Cache

Executes query to retrieve raw data

View Updates

Accounts for pending changes

Instantiates object faults



Persistence Framework



CoreData



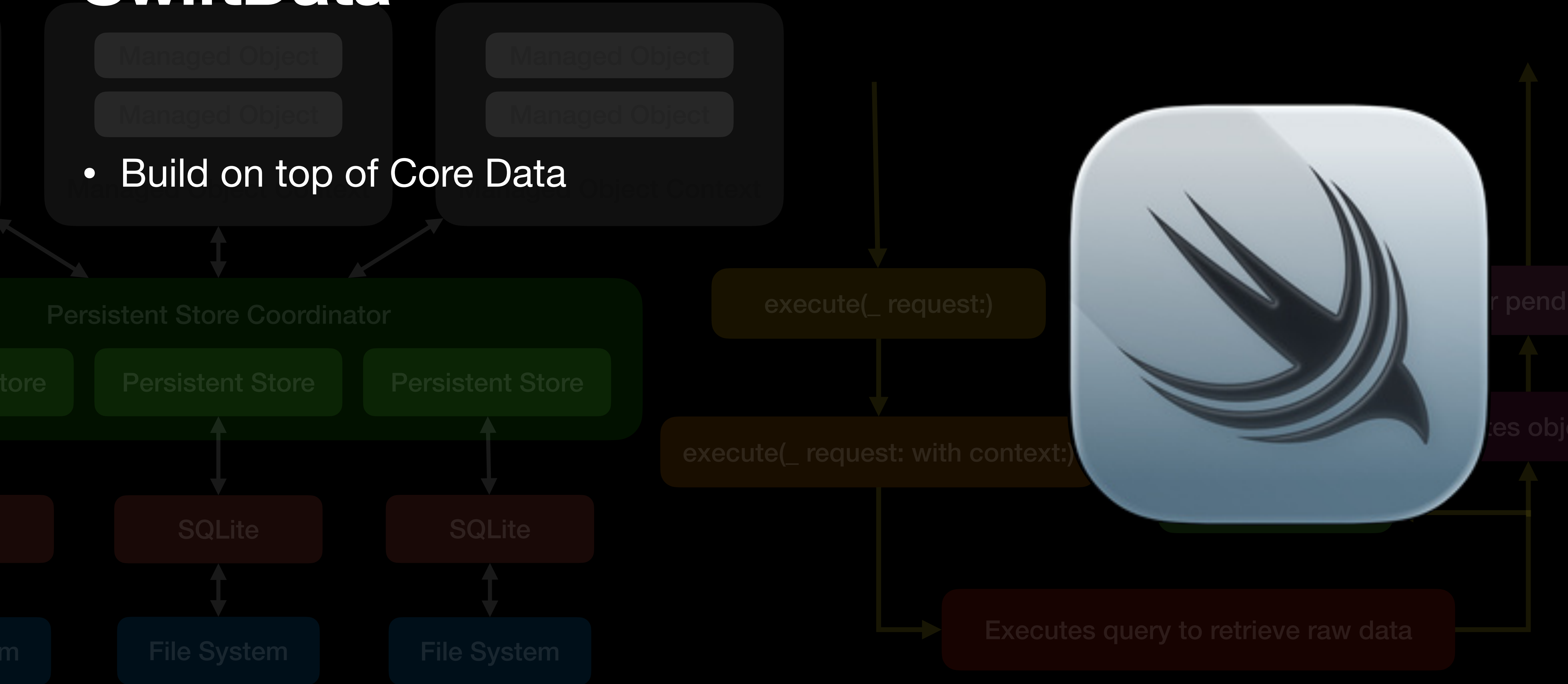
SwiftData



GRDB

SwiftData

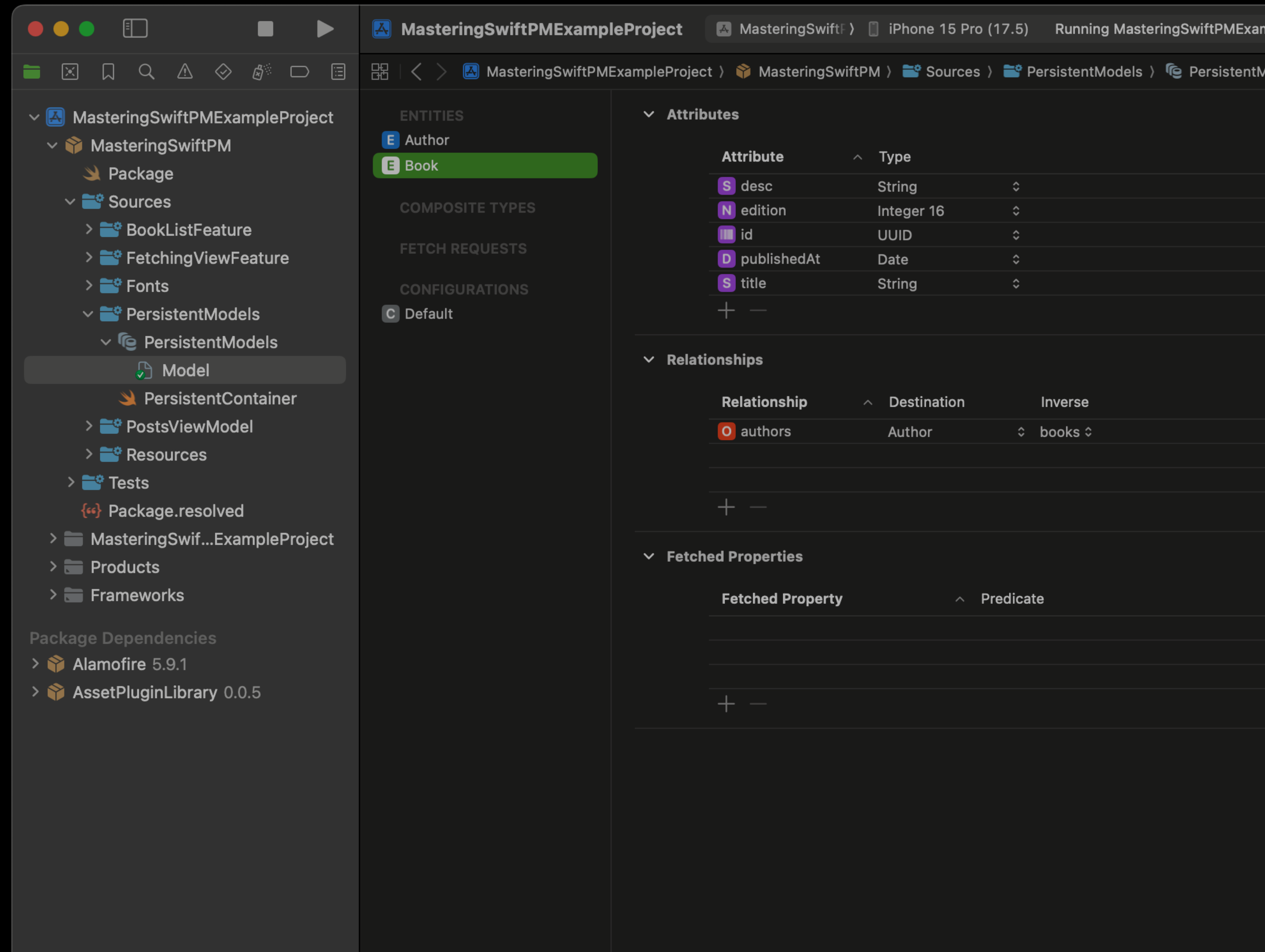
- Build on top of Core Data



SwiftData



- Build on top of Core Data
- No Xcode Entity Graphs



SwiftData



- Build on top of Core Data
- No Xcode Entity Graphs
- Swift Macro for reduced boilerplate
 - *@Model, @Query, @Attribute, @Relationship*



```
@Model
class Item {
    var title: String
    var isInStock: Bool
    var notes: String

    init(
        title: String = "",
        isInStock: Bool = true,
        notes: String = ""
    ) {
        self.title = title
        self.isInStock = isInStock
        self.notes = notes
    }
}
```

Model Definition

```
do {
    let modelContext = try ModelContext(.init(for: Item.self))
    self.modelContext = modelContext
} catch { fatalError() }
```

Context Initialisation

```
@main
struct SwiftDataStudyApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
                .modelContainer(for: Item.self)
        }
    }
}
```

Context Initialisation & Passing in Environment

```
func save(item: Item) throws {
    modelContext.insert(item)
    try modelContext.save()
}
```

Save Item

```
struct ContentView: View {
    @Query var items: [Item]

    var body: some View {
        List(items) { item in
            Text(item.title)
        }
    }
}
```

Read Items

SwiftData

Limitations



- Not All CoreData functionalities available
- Only Class types
- No NSCompoundPredicate
- No NSFetchedResultsController
- No support for Derived attributes
- No sectioned fetch requests
- No direct way to reload the query
- iOS 17+
- Enum support is buggy
- Limited support for CloudKit Syncing and Collaboration or Sharing

SQLiteData

SwiftData Alternative

SQLiteData

SwiftData Alternative

- Build on top of GRDB
- Allows Struct types
- Macros, Variadic Generics
- SQL Queries in Swift (Structured Queries)
- Supports Observation
- Operates on SQLite tier
- SQL Triggers
- Free* CloudKit sync & Collaboration/Sharing



SQLiteData

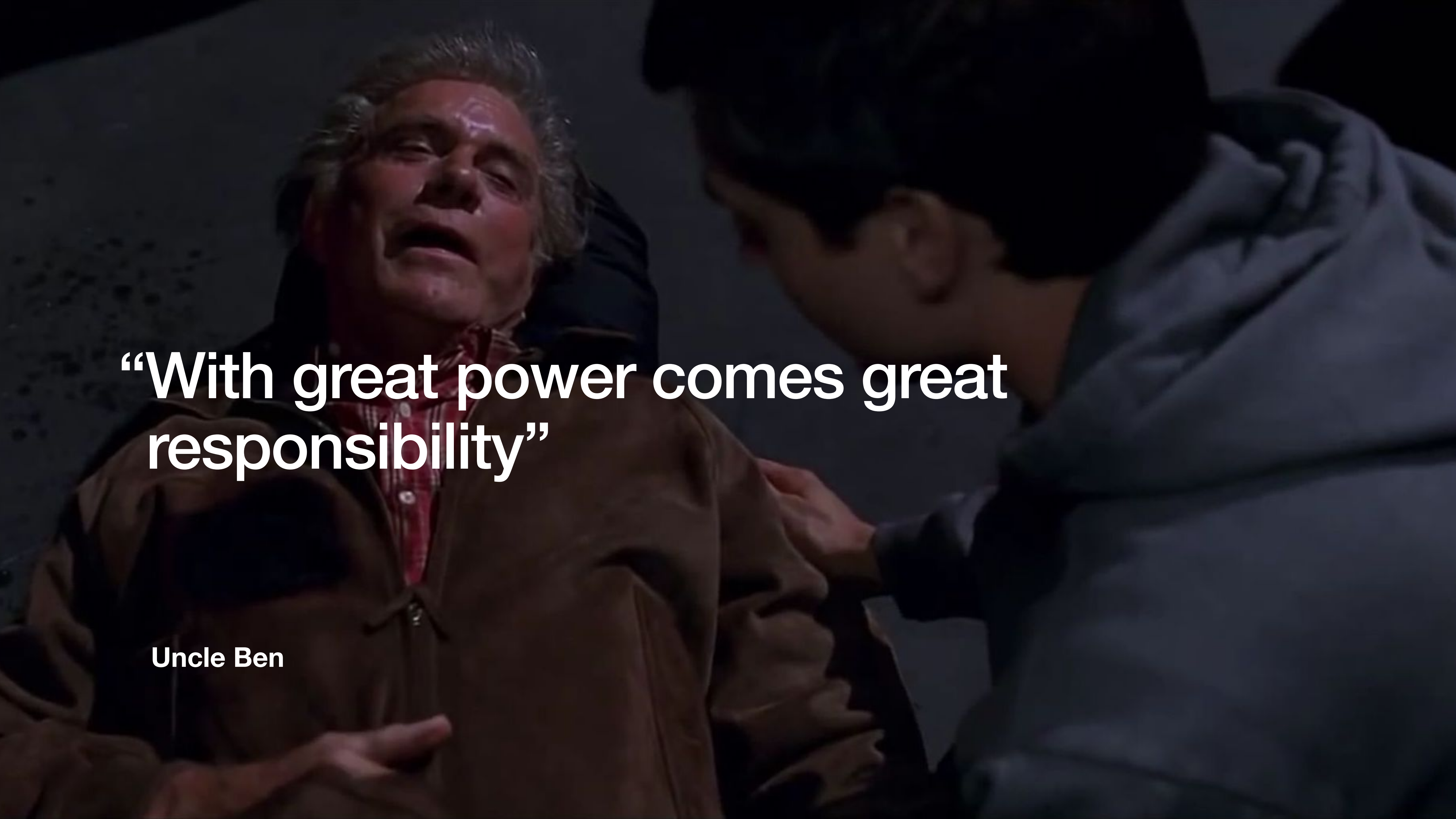
Similar APIs as SwiftData

```
@Table
struct AboutPage: Codable, Equatable, Identifiable, Sendable {
    var id: String
    var name: String
    var description: String
    var logo: String
}
```

```
@Dependency(\.defaultDatabase) var database
try database.read { db in
    try AboutPage
        .where {
            $0.name.collate(.nocase).contains("Swift")
        }
        .fetchAll(db)
}
```

```
struct AboutPages: View {
    @FetchAll(AboutPage.all) var aboutPages
    var body: some View {
        List(aboutPages) { aboutPage in
            Text(aboutPage.name)
        }
    }
}
```

```
func save(page: AboutPage) throws {
    @Dependency(\.defaultDatabase) var database
    try database.write { db in
        try AboutPage.insert { page }.execute(db)
    }
}
```

A still from the movie Spider-Man: Into the Spider-Verse. Uncle Ben is on the left, wearing a brown jacket over a red plaid shirt, looking up with a pained expression. Peter Parker is on the right, wearing a blue hoodie, looking down at Uncle Ben. The background is dark and blurry.

“With great power comes great responsibility”

Uncle Ben



SQLite

Knowledge Required

SQLiteData

Responsibilities

- Manual Table Creation
- Manual Migrations
- Manual Relationships
- Sometimes Raw SQL Queries
- Careful with SQL Triggers



SQLiteData

Migrations

```
@Table
struct AboutPage: Codable, Equatable, Identifiable, Sendable {
    var id: String
    var name: String
    var description: String
    var logo: String
}
```

```
public static func migrate(using migrator: inout DatabaseMigrator) throws {
    migrator.registerMigration("Create AboutPages Table") { database in
        try database.create(table: Self.tableName) { t in
            t.column("id", .text).primaryKey()
            t.column("name", .text).notNull()
            t.column("description", .text).notNull()
            t.column("logo", .text).notNull()
        }
    }
}
```

SQLiteData

Migrations

```
@Table
struct AboutPage: Codable, Equatable, Identifiable, Sendable {
    var id: String
    var name: String
    var description: String
    var logo: String
    var members: [String]
}
```

```
public static func migrate(using migrator: inout DatabaseMigrator) throws {
    migrator.registerMigration("Create AboutPages Table") { database in
        try database.create(table: Self.tableName) { t in
            t.column("id", .text).primaryKey()
            t.column("name", .text).notNull()
            t.column("description", .text).notNull()
            t.column("logo", .text).notNull()
        }
    }
}
```

SQLiteData

Migrations

```
@Table
struct AboutPage: Codable, Equatable, Identifiable, Sendable {
    var id: String
    var name: String
    var description: String
    var logo: String
    @Column(as: [String].JSONRepresentation.self)
    var members: [String]
}
```

```
static func migrate(using migrator: inout DatabaseMigrator) throws {
    migrator.registerMigration("Create AboutPages Table") { database in
        try database.create(table: Self.tableName) { t in
            t.column("id", .text).primaryKey()
            t.column("name", .text).notNull()
            t.column("description", .text).notNull()
            t.column("logo", .text).notNull()
        }
    }
}
```

SQLiteData

```
static func migrate(using migrator: inout DatabaseMigrator) throws {
    migrator.registerMigration("Create AboutPages Table") { database in
        try database.create(table: Self.tableName) { t in
            t.column("id", .text).primaryKey()
            t.column("name", .text).notNull()
            t.column("description", .text).notNull()
            t.column("logo", .text).notNull()
        }
    }
    migrator.registerMigration("Create Members column to AboutPages table") { database in
        try database.alter(table: Self.tableName) { t in
            t.add(column: "members", .jsonText).notNull().defaults(to: "[]")
        }
    }
}
```

SQLiteData

<pre>@Query var items: [Item]</pre>
<pre>@Query(sort: [SortDescriptor(\.title)]) var items: [Item]</pre>
<pre>@Query(filter: #Predicate<Item> { \$0.isInStock }) var items: [Item]</pre>
<pre>// No @Query equivalent of ordering // by boolean column.</pre>
<pre>// No @Query equivalent of counting // entries in database without loading // all entries.</pre>

SwiftData

<pre>@FetchAll var items: [Item]</pre>
<pre>@FetchAll(Item.order(by: \.title)) var items</pre>
<pre>@FetchAll(Item.where(\.isInStock)) var items</pre>
<pre>@FetchAll(Item.order(by: \.isInStock)) var items</pre>
<pre>@FetchOne(Item.count()) var itemCount = 0</pre>

SQLiteData

```
extension YearReviewItems {
  public static func items(for year: String) -> Select<YearReviewItems, Event, (Talk?, TalkSpeaker?, Speaker?)> {
    Event
      .where { _ in
        #sql("STRFTIME('%Y', events.'date') == \(bind: year)", as: Bool.self)
      }
      .leftJoin(Talk.all) { event, talk in
        event.id.eq(talk.eventID)
      }
      .leftJoin(TalkSpeaker.all) { event, talk, talkSpeaker in
        talk.id.eq(talkSpeaker.talkID)
      }
      .leftJoin(Speaker.all) { event, talk, talkSpeaker, speaker in
        talkSpeaker.speakerID.eq(speaker.id)
      }
      .select { event, talk, talkSpeaker, speaker in
        YearReviewItems.Columns(
          events: event.jsonGroupArray(distinct: true),
          talks: talk.jsonGroupArray(distinct: true, order: event.date.asc()),
          speakers: speaker.jsonGroupArray(distinct: true)
        )
      }
  }
}
```

```

@Selection
public struct YearReviewItems: Codable, Equatable, Sendable {
    @Column(as: [Event].JSONRepresentation.self)
    var events: [Event]
    @Column(as: [Talk].JSONRepresentation.self)
    public var talks: [Talk]
    @Column(as: [Speaker].JSONRepresentation.self)
    public var speakers: [Speaker]

    public init(...) {
        ...
    }
}

```

```

extension YearReviewItems {
    public static func items(for year: String) -> Select
        Event
        .where { _ in
            #sql("STRFTIME('%Y', events.'date') == \
        }
        .leftJoin(Talk.all) { event, talk in
            event.id.eq(talk.eventID)
        }
        .leftJoin(TalkSpeaker.all) { event, talk, ta
            talk.id.eq(talkSpeaker.talkID)
        }
        .leftJoin(Speaker.all) { event, talk, talkSp
            talkSpeaker.speakerID.eq(speaker.id)
        }
        .select { event, talk, talkSpeaker, speaker
            YearReviewItems.Columns(
                events: event.jsonGroupArray(distinct
                talks: talk.jsonGroupArray(distinct
                speakers: speaker.jsonGroupArray(dis
            )
        }
    }
}

```

```
11 @Selection
12 public struct YearReviewItems: Codable, Equatable, Sendable {
13     @Column(as: [Event].JSONRepresentation.self)
14     var events: [Event]
15     @Column(as: [Talk].JSONRepresentation.self)
16     public var talks: [Talk]
17     @Column(as: [Speaker].JSONRepresentation.self)
18     public var speakers: [Speaker]
19
20     public init(events: [Event] = [], talks: [Talk] = [], speakers: [Speaker] = []) {
21         self.events = events
22         self.talks = talks
23         self.speakers = speakers
24     }
25
26     public var speakerTexts: [String] {
27         self.speakers.map { $0.name }.joined(separator: " ").split(separator: " ").map { String($0) }
28     }
29
30     public var topicTexts: [String] {
31         self.talks.map { $0.title }.joined(separator: " ").split(separator: " ").map { String($0) }
32     }
```

```

11 @Selection
12 public struct YearReviewItems: Codable, Equatable, Sendable {
13     @Column(as: [Event].JSONRepresentation.self)
14     var events: [Event]
15     @Column(as: [Talk].JSONRepresentation.self)
16     public var talks: [Talk]
17     @Column(as: [Speaker].JSONRepresentation.self)
18     public var speakers: [Speaker]
19
20     public init(events: [Event] = [], talks: [Talk] = [], speakers: [Speaker] = []) {
21         self.events = events
22         self.talks = talks
23         self.speakers = speakers
24     }
25
26     public var speakerTexts: [String] {
27         self.speakers.map { $0.name }.joined(separator: " ").split(separator: " ").map { String($0) }
28     }
29
30     public var topicTexts: [String] {
31         self.talks.map { $0.title }.joined(separator: " ").split(separator: " ").map { String($0) }
32     }

```

```

× public nonisolated struct TableColumns: StructuredQueriesCore.TableDefinition {
2     public typealias QueryValue = YearReviewItems
3     public let events = StructuredQueriesCore.TableColumn<QueryValue, [Event].JSONRepresentation>("events", keyPath: \QueryValue.events)
4     public let talks = StructuredQueriesCore.TableColumn<QueryValue, [Talk].JSONRepresentation>("talks", keyPath: \QueryValue.talks)
5     public let speakers = StructuredQueriesCore.TableColumn<QueryValue, [Speaker].JSONRepresentation>("speakers", keyPath: \QueryValue.speakers)
6     public static var allColumns: [any StructuredQueriesCore.TableColumnExpression] {
7         var allColumns: [any StructuredQueriesCore.TableColumnExpression] = []
8         allColumns.append(contentsOf: QueryValue.columns.events._allColumns)
9         allColumns.append(contentsOf: QueryValue.columns.talks._allColumns)
10        allColumns.append(contentsOf: QueryValue.columns.speakers._allColumns)
11        return allColumns
12    }
13    public static var writableColumns: [any StructuredQueriesCore.WritableTableColumnExpression] {
14        var writableColumns: [any StructuredQueriesCore.WritableTableColumnExpression] = []
15        writableColumns.append(contentsOf: QueryValue.columns.events._writableColumns)
16        writableColumns.append(contentsOf: QueryValue.columns.talks._writableColumns)
17        writableColumns.append(contentsOf: QueryValue.columns.speakers._writableColumns)
18        return writableColumns
19    }

```

```

7     var allColumns: [any StructuredQueriesCore.TableColumnExpression] = []
8     allColumns.append(contentsOf: QueryValue.columns.events._allColumns)
9     allColumns.append(contentsOf: QueryValue.columns.talks._allColumns)
10    allColumns.append(contentsOf: QueryValue.columns.speakers._allColumns)
11    return allColumns
12 }
13 public static var writableColumns: [any StructuredQueriesCore.WritableTableColumnExpression] {
14     var writableColumns: [any StructuredQueriesCore.WritableTableColumnExpression] = []
15     writableColumns.append(contentsOf: QueryValue.columns.events._writableColumns)
16     writableColumns.append(contentsOf: QueryValue.columns.talks._writableColumns)
17     writableColumns.append(contentsOf: QueryValue.columns.speakers._writableColumns)
18     return writableColumns
19 }
20 public var queryFragment: QueryFragment {
21     "\(self.events), \(self.talks), \(self.speakers)"
22 }
23 }

```

```

24
25 public nonisolated struct Selection: StructuredQueriesCore.TableExpression {
26     public typealias QueryValue = YearReviewItems
27     public let allColumns: [any StructuredQueriesCore.QueryExpression]
28     public init(
29         events: some StructuredQueriesCore.QueryExpression<[Event].JSONRepresentation>,
30         talks: some StructuredQueriesCore.QueryExpression<[Talk].JSONRepresentation>,
31         speakers: some StructuredQueriesCore.QueryExpression<[Speaker].JSONRepresentation>
32     ) {
33         var allColumns: [any StructuredQueriesCore.QueryExpression] = []
34         allColumns.append(contentsOf: events._allColumns)
35         allColumns.append(contentsOf: talks._allColumns)
36         allColumns.append(contentsOf: speakers._allColumns)
37         self.allColumns = allColumns
38     }
39 }

```

```

33 }
34
35 ⌘ nonisolated extension YearReviewItems: StructuredQueriesCore.Table, StructuredQueriesCore._Selection, StructuredQueriesCore.PartialSelectStatement {
36     public typealias QueryValue = Self
37     public typealias From = Swift.Never
38     public nonisolated static var columns: TableColumns {
39         TableColumns()
40     }
41 }

```

```
29
30 public var topicTexts: [String] {
31     self.talks.map { $0.title }.joined(separator: " ").split(separator: " ").map { S
32 }

× public nonisolated struct TableColumns: StructuredQueriesCore.TableDefinition {
 2 public typealias QueryValue = YearReviewItems
 3 public let events = StructuredQueriesCore.TableColumn<QueryValue, [Event].JSONRepres
 4 public let talks = StructuredQueriesCore.TableColumn<QueryValue, [Talk].JSONRepres
 5 public let speakers = StructuredQueriesCore.TableColumn<QueryValue, [Speaker].JSONRe
 6 public static var allColumns: [any StructuredQueriesCore.TableColumnExpression] {
 7     var allColumns: [any StructuredQueriesCore.TableColumnExpression] = []
 8     allColumns.append(contentsOf: QueryValue.columns.events._allColumns)
 9     allColumns.append(contentsOf: QueryValue.columns.talks._allColumns)
10     allColumns.append(contentsOf: QueryValue.columns.speakers._allColumns)
11     return allColumns
12 }
13 public static var writableColumns: [any StructuredQueriesCore.WritableTableColumnExp
14     var writableColumns: [any StructuredQueriesCore.WritableTableColumnExpression] =
15     writableColumns.append(contentsOf: QueryValue.columns.events._writableColumns)
16     writableColumns.append(contentsOf: QueryValue.columns.talks._writableColumns)
17     writableColumns.append(contentsOf: QueryValue.columns.speakers._writableColumns)
18     return writableColumns
19 }
20 public var queryFragment: QueryFragment {
```

```
29
30 public var topicTexts: [String] {
31     self.talks.map { $0.title }.joined(separator: " ").split(separator: " ").map { S
32 }

× public nonisolated struct TableColumns: StructuredQueriesCore.TableDefinition {
2     public typealias QueryValue = YearReviewItems
3     public let events = StructuredQueriesCore.TableColumn<QueryValue, [Event].JSONRepres
4     public let talks = StructuredQueriesCore.TableColumn<QueryValue, [Talk].JSONRepreser
5     public let speakers = StructuredQueriesCore.TableColumn<QueryValue, [Speaker].JSONRe
6     public static var allColumns: [any StructuredQueriesCore.TableColumnExpression] {
7         var allColumns: [any StructuredQueriesCore.TableColumnExpression] = []
8         allColumns.append(contentsOf: QueryValue.columns.events._allColumns)
9         allColumns.append(contentsOf: QueryValue.columns.talks._allColumns)
10        allColumns.append(contentsOf: QueryValue.columns.speakers._allColumns)
11        return allColumns
12    }
13    public static var writableColumns: [any StructuredQueriesCore.WritableTableColumnExp
14        var writableColumns: [any StructuredQueriesCore.WritableTableColumnExpression] =
15        writableColumns.append(contentsOf: QueryValue.columns.events._writableColumns)
16        writableColumns.append(contentsOf: QueryValue.columns.talks._writableColumns)
17        writableColumns.append(contentsOf: QueryValue.columns.speakers._writableColumns)
18        return writableColumns
19    }
20    public var queryFragment: QueryFragment {
```

```
extension YearReviewItems {  
  public static func items(for year: String) -> Select<YearReviewItems, Event, (Talk?, TalkSpeaker?, Speaker?)> {  
    Event  
    .where { _ in  
      #sql("STRFTIME('%Y', events.date) == \$(bind: year)", as: Bool.self)  
    }  
    .leftJoin(Talk.all) { event, talk in  
      event.id.eq(talk.eventID)  
    }  
    .leftJoin(TalkSpeaker.all) { event, talk, talkSpeaker in  
      talk.id.eq(talkSpeaker.talkID)  
    }  
    .leftJoin(Speaker.all) { event, talk, talkSpeaker, speaker in  
      talkSpeaker.speakerID.eq(speaker.id)  
    }  
    .select { event, talk, talkSpeaker, speaker in  
      YearReviewItems.Columns {  
        events: event.jsonSupArray("distinct", true)  
        talks: talk.jsonGroupArray("distinct", true, order: event.date.asc()),  
        speakers: speaker.jsonGroupArray("distinct", true)  
      }  
    }  
  }  
}
```

All are

Table Columns

Types

```
extension YearReviewItems {
  public static func items(for year: String) -> Select<YearReviewItems, Event, (Talk?, TalkSpeaker?, Speaker?)> {
    Event
      .where { _ in
        #sql("STRFTIME('%Y', events.'date') == \(bind: year)", as: Bool.self)
      }
      .leftJoin(Talk.all) { event, talk in
        event.id.eq(talk.eventID)
      }
      .leftJoin(TalkSpeaker.all) { event, talk, talkSpeaker in
        talk.id.eq(talkSpeaker.talkID)
      }
      .leftJoin(Speaker.all) { event, talk, talkSpeaker, speaker in
        talkSpeaker.speakerID.eq(speaker.id)
      }
      .select { event, talk, talkSpeaker, speaker in
        YearReviewItems.Columns(
          events: event.jsonGroupArray(distinct: true),
          talks: talk.jsonGroupArray(distinct: true, order: event.date.asc()),
          speakers: speaker.jsonGroupArray(distinct: true)
        )
      }
  }
}
```

```
extension YearReviewItems {
  public static func items(for year: String) -> Select<YearReviewItems, Event, (Talk?, TalkSpeaker?, Speaker?)> {
    Event
      .where { _ in
        #sql("STRFTIME('%Y', events.'date') == \(bind: year)", as: Bool.self)
      }
      .leftJoin(Talk.all) { event, talk in
        event.id.eq(talk.eventID)
      }
      .leftJoin(TalkSpeaker.all) { event, talk, talkSpeaker in
        talk.id.eq(talkSpeaker.talkID)
      }
      .leftJoin(Speaker.all) { event, talk, talkSpeaker, speaker in
        talkSpeaker.speakerID.eq(speaker.id)
      }
      .select { event, talk, talkSpeaker, speaker in
        YearReviewItems.Columns(
          events: event.jsonGroupArray(distinct: true),
          talks: talk.jsonGroupArray(distinct: true, order: event.date.asc()),
          speakers: speaker.jsonGroupArray(distinct: true)
        )
      }
  }
}
```

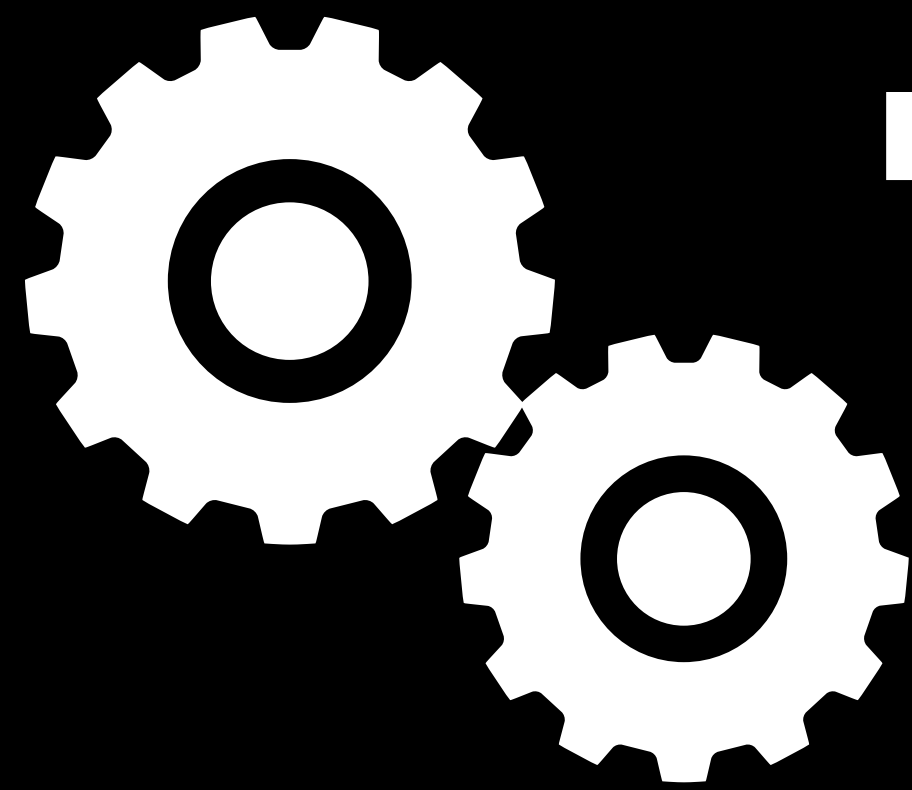
```
extension YearReviewItems {
  public static func items(for year: String) -> Select<YearReviewItems, Event, (Talk?, TalkSpeaker?, Speaker?)> {
    Event
      .where { _ in
        #sql("STRFTIME('%Y', events.'date') == \(bind: year)", as: Bool.self)
      }
      .leftJoin(Talk.all) { event, talk in
        event.id.eq(talk.eventID)
      }
      .leftJoin(TalkSpeaker.all) { event, talk, talkSpeaker in
        talk.id.eq(talkSpeaker.talkID)
      }
      .leftJoin(Speaker.all) { event, talk, talkSpeaker, speaker in
        talkSpeaker.speakerID.eq(speaker.id)
      }
      .select { event, talk, talkSpeaker, speaker in
        YearReviewItems.Columns(
          events: event.jsonGroupArray(distinct: true),
          talks: talk.jsonGroupArray(distinct: true, order: event.date.asc()),
          speakers: speaker.jsonGroupArray(distinct: true)
        )
      }
  }
}
```

```
extension YearReviewItems {
  public static func items(for year: String) -> Select<YearReviewItems, Event, (Talk?, TalkSpeaker?, Speaker?)> {
    Event
    .where { _ in
      #sql("STRFTIME('%Y', events.'date') == \(bind: year)", as: Bool.self)
    }
    .leftJoin(Talk.all) { event, talk in
      event.id.eq(talk.eventID)
    }
    .leftJoin(TalkSpeaker.all) { event, talk, talkSpeaker in
      talk.id.eq(talkSpeaker.talkID)
    }
    .leftJoin(Speaker.all) { event, talk, talkSpeaker, speaker in
      talkSpeaker.speakerID.eq(speaker.id)
    }
    .select { event, talk, talkSpeaker, speaker in
      YearReviewItems.Columns(
        events: event.jsonGroupArray(distinct: true),
        talks: talk.jsonGroupArray(distinct: true, order: event.date.asc()),
        speakers: speaker.jsonGroupArray(distinct: true)
      )
    }
  }
}
```

```
extension YearReviewItems {
  public static func items(for year: String) -> Select<YearReviewItems, Event, (Talk?, TalkSpeaker?, Speaker?)> {
    Event
    .where { _ in
      #sql("STRFTIME('%Y', events.'date') == \(bind: year)", as: Bool.self)
    }
    .leftJoin(Talk.all) { event, talk in
      event.id.eq(talk.eventID)
    }
    .leftJoin(TalkSpeaker.all) { event, talk, talkSpeaker in
      talk.id.eq(talkSpeaker.talkID)
    }
    .leftJoin(Speaker.all) { event, talk, talkSpeaker, speaker in
      talkSpeaker.speakerID.eq(speaker.id)
    }
    .select { event, talk, talkSpeaker, speaker in
      YearReviewItems.Columns(
        events: event.jsonGroupArray(distinct: true),
        talks: talk.jsonGroupArray(distinct: true, order: event.date.asc()),
        speakers: speaker.jsonGroupArray(distinct: true)
      )
    }
  }
}
```

```
SELECT json_group_array(DISTINCT json_object('id', json_quote("events"."id"), 'communityID',  
json_quote("events"."communityID"), 'title', json_quote("events"."title"), 'date', json_quote("events"."date"),  
'endDate', json_quote("events"."endDate"))) AS "events", json_group_array(DISTINCT CASE WHEN ("talks"."rowid") IS  
NOT (NULL) THEN json_object('id', json_quote("talks"."id"), 'title', json_quote("talks"."title"), 'eventID',  
json_quote("talks"."eventID")) END ORDER BY "events"."date" ASC) FILTER (WHERE ("talks"."rowid") IS NOT (NULL)) AS  
"talks", json_group_array(DISTINCT CASE WHEN ("speakers"."rowid") IS NOT (NULL) THEN json_object('id',  
json_quote("speakers"."id"), 'name', json_quote("speakers"."name"), 'socials', json("speakers"."socials"), 'about',  
json_quote("speakers"."about"), 'image', json_quote("speakers"."image")) END) FILTER (WHERE ("speakers"."rowid") IS  
NOT (NULL)) AS "speakers" FROM "events" LEFT JOIN "talks" ON ("events"."id") = ("talks"."eventID") LEFT JOIN  
"talkSpeakers" ON ("talks"."id") = ("talkSpeakers"."talkID") LEFT JOIN "speakers" ON ("talkSpeakers"."speakerID") =  
("speakers"."id") WHERE (STRFTIME('%Y', events.'date') == '2025')
```

Generated SQL Query



Triggers

SQL Triggers



Practice Speech

Audio Recording & Rehearsal

- Record Speech
- Listen it back
- Enhance speech delivery
- Remember complicated words



Practice Speech

Audio Recording & Rehearsal

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    var id: UUID
    var title: String
    var createdAt: Date
    var storagePath: String
    var duration: TimeInterval

    var storageURL: URL {
        URL.documentsDirectory.appending(path: storagePath)
    }
}
```



Practice Speech

Audio Recording & Rehearsal

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    var id: UUID
    var title: String
    var createdAt: Date
    var storagePath: String
    var duration: TimeInterval

    var storageURL: URL {
        URL.documentsDirectory.appending(path: storagePath)
    }
}
```

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    ...
}
```

```
extension PracticeSpeech {
    public static func registerTriggers(in database: any DatabaseWriter) throws {
        try database.write { db in
            try PracticeSpeech.createTemporaryTrigger after: .delete(forEachRow: { old in
                #sql("SELECT \($deletePracticeSpeechStorage(path: old.storagePath))")
            })
            .execute(db)
        }
    }
}
```

```
@DatabaseFunction()
public func deletePracticeSpeechStorage(path: String) {
    let fileURL = URL.documentsDirectory.appending(path: path)
    try? FileManager.default.removeItem(at: fileURL)
}
```

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    ...
}
```

```
extension PracticeSpeech {
    public static func registerTriggers(in database: any DatabaseWriter) throws {
        try database.write { db in
            try PracticeSpeech.createTemporaryTrigger(after: .delete(forEachRow: { old in
                #sql("SELECT \($deletePracticeSpeechStorage(path: old.storagePath))")
            }))
            .execute(db)
        }
    }
}
```

```
@DatabaseFunction()
public func deletePracticeSpeechStorage(path: String) {
    let fileURL = URL.documentsDirectory.appending(path: path)
    try? FileManager.default.removeItem(at: fileURL)
}
```

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    ...
}
```

```
extension PracticeSpeech {
    public static func registerTriggers(in database: any DatabaseWriter) throws {
        try database.write { db in
            try PracticeSpeech.createTemporaryTrigger(after: .delete(forEachRow: { old in
                #sql("SELECT \($deletePracticeSpeechStorage(path: old.storagePath))")
            }))
            .execute(db)
        }
    }
}
```

```
@DatabaseFunction()
public func deletePracticeSpeechStorage(path: String) {
    let fileURL = URL.documentsDirectory.appending(path: path)
    try? FileManager.default.removeItem(at: fileURL)
}
```

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    ...
}
```

```
extension PracticeSpeech {
    public static func registerTriggers(in database: any DatabaseWriter) throws {
        try database.write { db in
            try PracticeSpeech.createTemporaryTrigger(after: .delete(forEachRow: { old in
                #sql("SELECT \($deletePracticeSpeechStorage path: old.storagePath)")
            }))
            .execute(db)
        }
    }
}
```

```
@DatabaseFunction()
public func deletePracticeSpeechStorage(path: String) {
    let fileURL = URL.documentsDirectory.appending(path: path)
    try? FileManager.default.removeItem(at: fileURL)
}
```

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    ...
}
```

```
extension PracticeSpeech {
    public static func registerTriggers(in database: any DatabaseWriter) throws {
        try database.write { db in
            try PracticeSpeech.createTemporaryTrigger(after: .delete(forEachRow: { old in
                #sql("SELECT \($deletePracticeSpeechStorage(path: old.storagePath))")
            }))
            .execute(db)
        }
    }
}
```

```
@DatabaseFunction()
public func deletePracticeSpeechStorage(path: String) {
    let fileURL = URL.documentsDirectory.appending(path: path)
    try? FileManager.default.removeItem(at: fileURL)
}
```

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    ...
}
```

```
extension PracticeSpeech {
    public static func registerTriggers(in database: any DatabaseWriter) throws {
        try database.write { db in
            try PracticeSpeech.createTemporaryTrigger(after: .delete(forEachRow: { old in
                #sql("SELECT \($deletePracticeSpeechStorage(path: old.storagePath))")
            }))
            .execute(db)
        }
    }
}
```

```
@DatabaseFunction()
public func deletePracticeSpeechStorage(path: String) {
    let fileURL = URL.documentsDirectory.appending(path: path)
    try? FileManager.default.removeItem(at: fileURL)
}
```

```
@Table
struct PracticeSpeech: Codable, Equatable, Identifiable, Sendable {
    ...
}
```

```
extension PracticeSpeech {
    public static func registerTriggers(in database: any DatabaseWriter) throws {
        try database.write { db in
            try PracticeSpeech.createTemporaryTrigger(after: .delete(forEachRow: { old in
                #sql("SELECT \($deletePracticeSpeechStorage(path: old.storagePath))")
            }))
            .execute(db)
        }
    }
}
```

```
@DatabaseFunction()
public func deletePracticeSpeechStorage(path: String) {
    let fileURL = URL.documentsDirectory.appending(path: path)
    try? FileManager.default.removeItem(at: fileURL)
}
```

SQL Triggers

Usage

- Logging
- Data Validation
- Invoke external functions
- Synchronization
- Auto Time Stamps. i.e., createdAt, updatedAt

SQL Triggers

Usage

- Logging
- Data Validation
- Invoke external functions
- Synchronization
- Auto Time Stamps. i.e., createdAt, updatedAt



CloudKit Sync

CloudKit Sync

```
@main
struct MyApp: App {
    init() {
        prepareDependencies {
            $0.defaultDatabase = try! appDatabase()
            $0.defaultSyncEngine = SyncEngine(
                for: $0.defaultDatabase,
                tables: Item.self
            )
        }
    }
}
```



CloudKit Sync

```
@main
struct MyApp: App {
    init() {
        prepareDependencies {
            $0.defaultDatabase = try! appDatabase()
            $0.defaultSyncEngine = SyncEngine(
                for: $0.defaultDatabase,
                tables: Item.self
            )
        }
    }
}
```



CloudKit Sync

```
@main
struct MyApp: App {
    init() {
        prepareDependencies {
            $0.defaultDatabase = try! appDatabase()
            $0.defaultSyncEngine = SyncEngine(
                for: $0.defaultDatabase,
                tables: Item.self
            )
        }
    }
}
```



CloudKit Sync

```
@main
struct MyApp: App {
    init() {
        prepareDependencies {
            $0.defaultDatabase = try! appDatabase()
            $0.defaultSyncEngine = SyncEngine(
                for: $0.defaultDatabase,
                tables: Item.self
            )
        }
    }
}
```



CloudKit Sync

CloudKit Rules

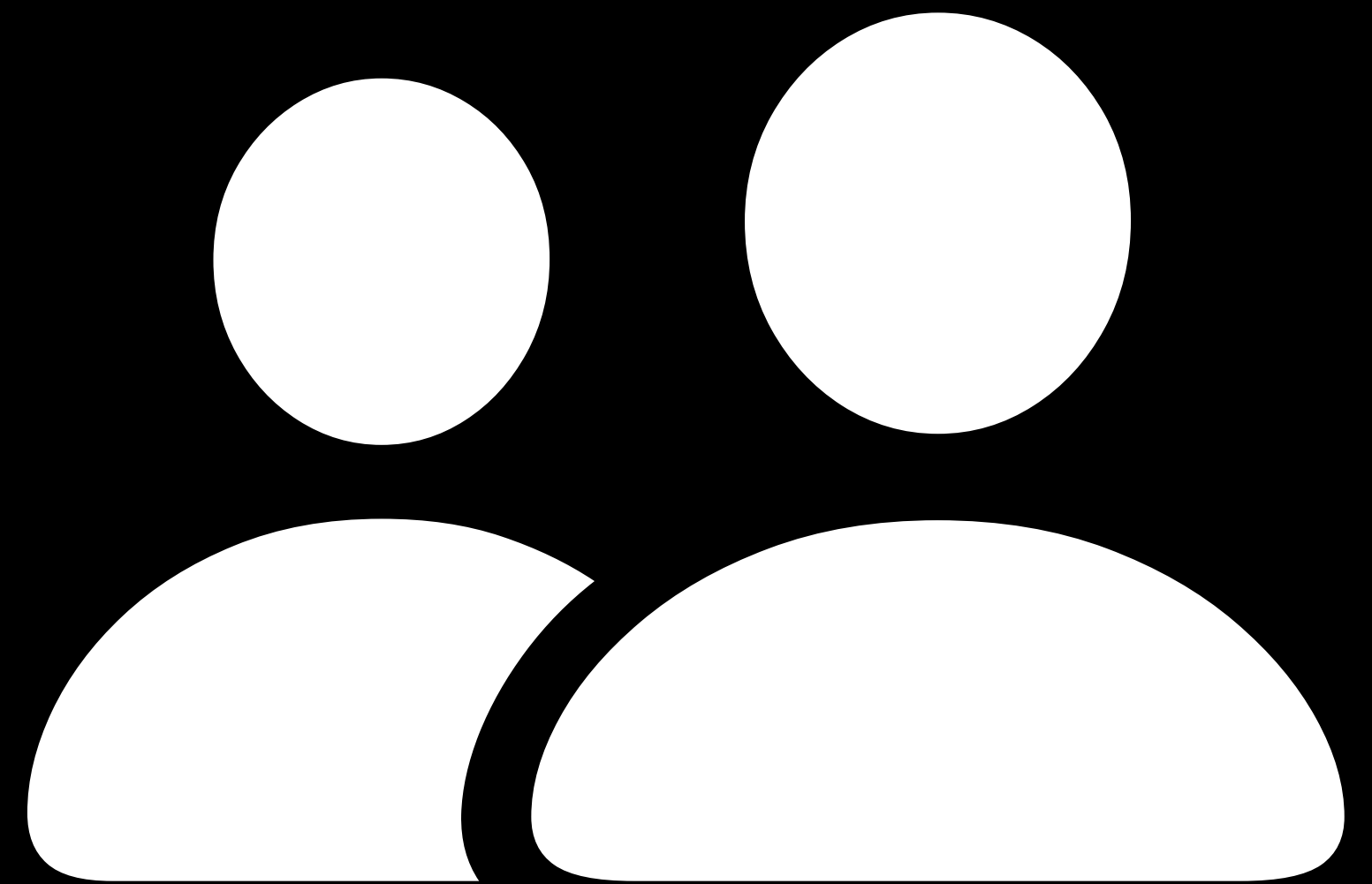
- No Int based Identifiers
- Only Additions, No Deletion (in CloudKit db)
- Explicit Relationship Delete Rules



CloudKit Sharing

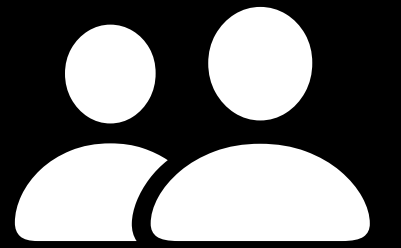
Sharing Rules

- Only Root Records can be shared
- Root Records = No Foreign Key Relation
- No Many-to-many relationships



CloudKit Collaboration

Sharing



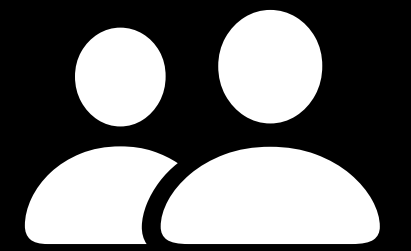
```
@Dependency(\.defaultSyncEngine) private var syncEngine
```

```
let share = try await syncEngine.share(record: community) { share in  
    share[CKShare.SystemFieldKey.title] = "Join \(community.name) to collaborate."  
    share[CKShare.SystemFieldKey.thumbnailImageData] = imageData  
}
```

```
CloudSharingView(sharedRecord: share)
```

CloudKit Collaboration

Sharing



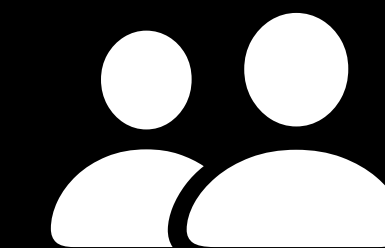
```
@Dependency(\.defaultSyncEngine) private var syncEngine
```

```
let share = try await syncEngine.share(record: community) { share in  
    share[CKShare.SystemFieldKey.title] = "Join \(community.name) to collaborate."  
    share[CKShare.SystemFieldKey.thumbnailImageData] = imageData  
}
```

```
CloudSharingView(sharedRecord: share)
```

CloudKit Collaboration

Sharing



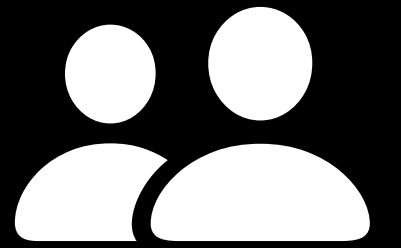
```
@Dependency(\.defaultSyncEngine) private var syncEngine
```

```
let share = try await syncEngine.share(record: community) { share in  
    share[CKShare.SystemFieldKey.title] = "Join \((community.name) to collaborate."  
    share[CKShare.SystemFieldKey.thumbnailImageData] = imageData  
}
```

```
CloudSharingView(sharedRecord: share)
```

CloudKit Collaboration

Sharing



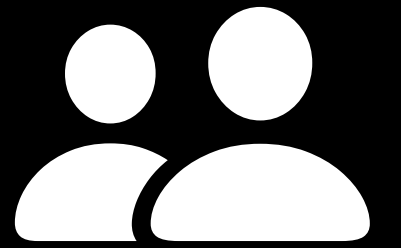
```
@Dependency(\.defaultSyncEngine) private var syncEngine
```

```
let share = try await syncEngine.share(record: community) { share in  
    share[CKShare.SystemFieldKey.title] = "Join \((community.name) to collaborate."  
    share[CKShare.SystemFieldKey.thumbnailImageData] = imageData  
}
```

```
CloudSharingView(sharedRecord: share)
```

CloudKit Collaboration

Sharing



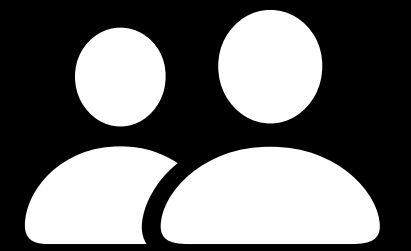
```
@Dependency(\.defaultSyncEngine) private var syncEngine
```

```
let share = try await syncEngine.share(record: community) { share in  
    share[CKShare.SystemFieldKey.title] = "Join \(community.name) to collaborate."  
    share[CKShare.SystemFieldKey.thumbnailImageData] = imageData  
}
```

```
CloudSharingView(sharedRecord: share)
```

CloudKit Collaboration

Sharing



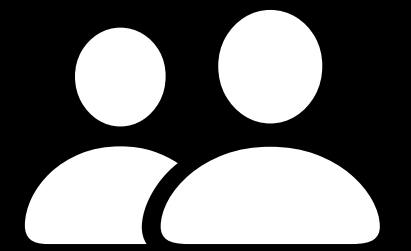
```
@Dependency(\.defaultSyncEngine) private var syncEngine
```

```
let share = try await syncEngine.share(record: community) { share in  
    share[CKShare.SystemFieldKey.title] = "Join \((community.name) to collaborate."  
    share[CKShare.SystemFieldKey.thumbnailImageData] = imageData  
}
```

```
CloudSharingView(sharedRecord: share)
```

CloudKit Collaboration

Sharing



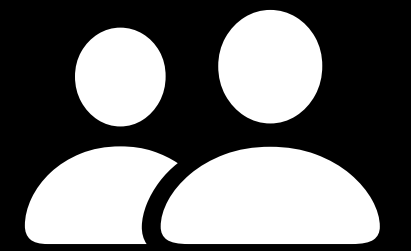
```
@Dependency(\.defaultSyncEngine) private var syncEngine
```

```
let share = try await syncEngine.share(record: community) { share in  
    share[CKShare.SystemFieldKey.title] = "Join \$(community.name) to collaborate."  
    share[CKShare.SystemFieldKey.thumbnailImageData] = imageData  
}
```

```
CloudSharingView(sharedRecord: share)
```

CloudKit Collaboration

Accepting Share



```
@Dependency(\.defaultSyncEngine) var syncEngine
```

```
func scene(
    _ scene: UIScene,
    willConnectTo session: UISceneSession,
    options connectionOptions: UIScene.ConnectionOptions
) {
    if let metadata = connectionOptions.cloudKitShareMetadata {
        try await syncEngine.acceptShare(metadata: metadata)
    }
}
```

```
func windowScene(
    _ windowScene: UIWindowScene,
    userDidAcceptCloudKitShareWith cloudKitShareMetadata: CKShare.Metadata) {
    try await syncEngine.acceptShare(metadata: metadata)
}
```

AI Era

“Skill” is all you need

The Point-Free Way

Expert-crafted AI skill documents for building long-lasting Swift applications.

Design, test and evolve applications using the same principles, libraries, and techniques we use every day at Point-Free.

Become a member to unlock

Explore Point-Free



Terminal

```
$ brew install pointfreeco/tap/pfw
$ pfw login
$ pfw install
$ ls -R ~/.codex/skills/
./ComposableArchitecture/
  SKILL.md

./SQLiteData/
  SKILL.md

./Dependencies/
  SKILL.md

./SwiftNavigation/
  SKILL.md
```

Demo

Today

Offline Apps using SQLiteData

- Database as Single Source of Truth
- CoreData vs SwiftData vs SQLiteData
- SQLite knowledge is Required
- Manual Migration
- Structured Queries -> Raw SQL
- SQL Triggers
- CloudKit Sync and Collaboration
- PFW Skills helps

Thank You

